
Leveraging Environment Interaction for Automated PDDL Translation and Planning with Large Language Models

Sadegh Mahdavi*
University of British Columbia
smahdavi@ece.ubc.ca

Raquel Aoki
Borealis AI
raquel.aoki@borealisai.com

Keyi Tang
Borealis AI
keyi.tang@borealisai.com

Yanshuai Cao
Borealis AI
yanshuai.cao@borealisai.com

Abstract

Large Language Models (LLMs) have shown remarkable performance in various natural language tasks, but they often struggle with planning problems that require structured reasoning. To address this limitation, the conversion of planning problems into the Planning Domain Definition Language (PDDL) has been proposed as a potential solution, enabling the use of automated planners. However, generating accurate PDDL files typically demands human inputs or correction, which can be time-consuming and costly. In this paper, we propose a novel approach that leverages LLMs and environment feedback to automatically generate PDDL domain and problem description files without the need for human intervention. Our method introduces an iterative refinement process that generates multiple problem PDDL candidates and progressively refines the domain PDDL based on feedback obtained from interacting with the environment. To guide the refinement process, we develop an Exploration Walk (EW) metric, which provides rich feedback signals for LLMs to update the PDDL file. We evaluate our approach on 10 PDDL environments. We achieve an average task solve rate of 66% compared to a 29% solve rate by GPT-4’s intrinsic planning with chain-of-thought prompting. Our work enables the automated modeling of planning environments using LLMs and environment feedback, eliminating the need for human intervention in the PDDL translation process and paving the way for more reliable LLM agents in challenging problems. Our code is available at <https://github.com/BorealisAI/llm-pddl-planning>

1 Introduction

Large language models (LLMs) have demonstrated remarkable success across various domains, including mathematics, coding, and even the bar exam [1]. These models excel at understanding and generating natural language, offering flexibility and adaptability to a wide range of tasks. However, when it comes to planning and long-horizon reasoning, LLMs have shown limited performance [8, 28], despite some promising results [3].

Planning is a crucial aspect of intelligence that involves reasoning to find a sequence of actions to achieve a desired goal state from an initial state. The Planning Domain Definition Language (PDDL) [18] is a widely used formalism for describing planning problems. PDDL provides a structured way

*Work performed while interning at Borealis AI.

to define the domain, which includes the types of objects, predicates, and actions, as well as the problem instance, which specifies the initial state and goal conditions. PDDL enables the application of search-based algorithms, such as breadth-first search (BFS) or A* search, which can guarantee to find a valid solution if one exists. However, the downside of PDDL is that it requires a well-defined and structured domain and problem definition, which can be challenging to create, especially for complex scenarios. Figure 1 showcases snippets of some PDDL problems and domain files along with an action plan produced by a classical planner.

Recent studies explored combining the strengths of LLMs and PDDL-based planning [15, 7, 9]. The idea is to leverage LLM for translation from natural language (NL) problem descriptions into PDDL formal descriptions, and then use a classical planner to solve the translated PDDL problem [9]. This hybrid approach could theoretically take advantage of the flexibility of NL input and the correctness guarantees provided by the classical planner. If the translation from NL to PDDL is accurate, the resulting plan is guaranteed to be valid.

Unfortunately, existing approaches have not been able to generate both PDDL problem and domain descriptions with reasonable success rates without humans in the loop, as we shall elaborate in Sec. 2. While translating PDDL problems is feasible given the domain PDDL description [15], generating domain PDDL from NL correctly is a more nuanced and challenging problem. To do so requires identifying causally relevant objects to design predicates, as well as their inter-relationships, in a way that accurately reflects the possible states and transitions of the environment. A small error, for example in predicate design, could lead to entirely incorrect domain description and failed planning (see Appendix A.2 for a real example). Guan et al. [9] take a step toward this goal relying on human-in-the-loop to detect and correct mistakes made by LLMs.

In this work, we develop a fully automated method for generating PDDL domain and problem definitions using LLMs and environment feedback without relying on human intervention. Intuitively, our method lets an LLM build hypothetical “mental models” of the environment, in the form of proposed PDDL domain descriptions. The LLM then verifies and updates the “mental model” by observing discrepancies between the feasibility of actions under its “mental model” and the real environment. This method enables LLMs to use classical planners to solve complex planning problems whose solutions may require hundreds or thousands of steps that all need to be correct.

We first highlight the challenges of this task and then propose our solution. In particular, our contributions are as follows:

- We demonstrate that even small modifications to PDDL domains can render plan search infeasible, limiting the feedback information for LLMs to perform in context update.
- To address this, we introduce a new Exploration Walk (EW) metric, which is a smooth similarity measure between two domains by comparing the executability of random action sequences sampled from one domain on the other. Crucially, EW only requires access to the action interface and executability of the environments, not directly the ground-truth PDDL.
- We propose an EW-guided tree search approach that leverages LLMs to generate and refine the PDDL domain and problem files iteratively and automatically.
- We evaluate our method on 10 challenging PDDL domains, where a number of them are from the International Planning Competition, and show that it outperforms a baseline that generates PDDL files in a single attempt without refinement. Our method solves 7 out of 10 environments, achieving an average task solve rate of 66% and average EW score of 0.84, compared to 34% task solve rate and 0.53 EW score for the baseline, and 29% solve rate by GPT-4 (gpt-4-1106-preview)’s intrinsic planning with chain-of-thought prompting.

To the best of our knowledge, this is the first work that enables modeling a planning environment via PDDL translation using LLMs and environment interaction, without the need for human intervention.

2 Related Work

LLMs and Classical Planning. There has been recent interest in integrating LLMs with PDDL [15, 28, 9, 7, 30, 23, 10, 20, 26], and more generally neural networks with PDDL [24, 2]. Silver et al. [25] leverage LLMs to take domain PDDLs and problem PDDL specifications, and synthesize a Python function to generate domain-specific plans, as a replacement for search-based planning. Liu

Table 1: Summary of comparison to most closely related prior studies.*Require at least one problem instance to be translated by a human into the target domain as an in-context example.

Method(s)	Translate Problem	Translate Domain	No Human Intervention
LLM+P [15], LLM-DP [7]	✓*	×	✓
LLM World Models [9]	✓	✓	×
Ours	✓	✓	✓

et al. [15] show that using LLMs to translate problem specification to PDDL, and using classical solvers results into a higher planning accuracy than using LLM directly as a planner. Dagan et al. [7] consider a similar setting, but assume that the list of objects is partially observable, and the LLM needs to interact with the world to observe the list of objects. All of the mentioned works, however, assume that a domain PDDL file is already provided. Oswald et al. [20] generate domain PDDL from natural language and propose heuristics for comparing PDDL action domains. However, their approach assumes that predicates are provided, whereas our work makes no such assumption. Additionally, Oswald et al. [20] rely on ground-truth problem instances for domain compatibility evaluation, whereas we directly translate problem PDDL without any such assumptions. Guan et al. [9] translate both Domain and Problem from natural language description but rely on human experts to correct mistakes in the domain translation before generating problem PDDLs. In this work, our goal is to lift the human-intervention assumption, and instead, use domain interaction for evaluation and verification. See Table 1 for a summary of related work comparison.

Direct Reasoning with LLMs. Recent research has explored eliciting direct reasoning capabilities within Large Language Models (LLMs). This reasoning can be either entirely direct [31, 29] or partially direct with the assistance of basic external tools [16]. However, the primary limitation of these approaches lies in the inherent tendency of auto-regressive LLMs to produce errors in long-horizon reasoning tasks [28]. Even a minor mistake in a single reasoning step can lead to cascading errors, ultimately resulting in an incorrect final answer [8]. When applied to classical planning, this approach delegates the entire plan generation process to an LLM instead of leveraging a dedicated classical planner. Studies have demonstrated that this strategy is suboptimal compared to generating PDDL code directly [9, 15], highlighting the importance of incorporating classical planning tools for faithful plan generation in classical planning tasks.

External Reasoning and Code Generation. This last line of work focuses on generating executable code from natural language instructions such as SQL or Python code generation [4, 19, 17, 5, 16, 32]. Here, the LLM often acts as a code translator, and the reasoning logic lies within the generated code. Chen et al. [4] show that LLMs are capable of Python code generation from docstrings to high accuracy. The authors also find that taking multiple code samples from an LLM and picking the best samples results in an accuracy boost. Later works show that iterative refinement of LLM responses improves the accuracy on the downstream task [17, 5], especially given external feedback such as unit tests or human feedback. Our work is related to code generation as we produce structured PDDL files. However, our setting presents three challenges: (1) there are two types of PDDL files, in contrast to a single Python script, and the two files need to be consistent with each other; (2) more importantly, getting external feedback and the evaluation of a generated PDDL code is not as easy as python unit tests, and as we show in Section 4.3, (domain generation) errors are abundant and hard to trace; (3) LLMs are trained with a lot more Python code compared to PDDL, as the latter is much scarcer.

3 Notation and Background

Notation. We denote $\mathbb{1}[\cdot]$ as the indicator function. The notation $1 : N$ refers to the sequence of integers ranging from 1 to N . For a set $\mathcal{A}$, we define $\mathcal{A}^*$ as the set comprising all possible sequences of elements drawn from $\mathcal{A}$, and define $2^{\mathcal{A}}$ as the power set of $\mathcal{A}$.

PDDL. Planning Domain Definition Language (PDDL) is a formal language used to describe and specify planning problems for automated planning. Here, we have two types of PDDL files: (1) *Domain PDDL*, which defines possible *predicates* (i.e., states), and *actions* in the environment. Executing each action requires some *precondition* (i.e., a set of predicates to have a specific value), and the execution leads to some *effect* (i.e., a change in the values of some predicates). (2) *Problem PDDL*, which contains a set of initial predicates and a set of goal predicates.

The problem PDDL instantiates the domain definition PDDL to form a concrete environment. Together, the planning problem is fully defined and formalized. A *classical planner* takes in both files and searches for a plan based on the provided specification. A *plan* is a sequence of actions, starting from the initial state, leading to a state satisfying the goal conditions, with each action respecting the rules of the environment. Formally, let $\mathcal{D}, \mathcal{P}, \mathcal{A}$ be the set of all possible domains, problems, and actions, respectively. Then, given a domain $d \in \mathcal{D}$ and problem $p \in \mathcal{P}$, a classical planner $C : \mathcal{D} \times \mathcal{P} \rightarrow \mathcal{A}^* \cup \{\perp\}$ takes in domain d and plan p , and produces a plan $q := C(d, p)$ which is either set of actions from $\mathcal{A}^*$, or a planning error $\perp$. A planning error may be due to an infeasible plan search (i.e., plan not found), syntax errors, or incompatible domain and problem. A plan validator verifies whether a plan q is executable and achieves the desired problem goal given a domain PDDL d and problem PDDL p , i.e., whether q solves the planning problem instance. The validator function, denoted as $V_{d,p}(q) : \mathcal{A}^* \rightarrow \{0, 1\}$, is 1 if the plan is valid, and 0 otherwise. For convenience, we assume $V_{d,p}(\perp) = 0$. Similarly, we define plan execution checker $E_{d,p} : \mathcal{A}^* \rightarrow \{0, 1\}$, which only checks whether an action sequence is executable in a domain or not. Note that the *difference between V and E is that the former checks for both plan executability and goal satisfaction, while the latter only checks for plan executability*. We also define $\mathcal{S}$ as the set of all possible states. Function $A_{d,p} : \mathcal{S} \rightarrow 2^{\mathcal{A}}$ delineates the set of legal actions given the current states (i.e., actions that would not immediately result in $E_{d,p}$ returning 0). The function $S_{d,p} : \mathcal{A} \times \mathcal{S} \rightarrow \mathcal{S}$ denotes the state transition function (i.e., $S_{d,p}(a, s)$ determines the subsequent state given the current state s and action a). Finally, we denote the initial state induced by d and p to be $s_{d,p,0} \in \mathcal{S}$. See Table 3 in the Appendix for a summary of notations.

To illustrate the definitions with an example, consider the Grippers [13] environment with several rooms containing robots and boxes. Robots can move balls between rooms using their left and right grippers. Given an initial setting of robots and balls in different rooms, the main goal is to move specific balls to specific rooms using the robots. Figure 1 shows an annotated example domain, problem, and plan for this environment. The domain determines predicates and actions. Predicates such as `at-robby` keep track of object states (e.g., whether a particular robot is in a particular room) and defining suitable predicates is a crucial part of domain design. The

move action for moving a robot from one room to another has three parameters: robot r , departure room $from$, and destination room to . Each action has preconditions and effects, which comprise the main logic of the domain for determining the actionability of an action. In the case of the move action, the precondition is that the robot must be in the `from` room, and the effect is that it will no longer be in that room and will be in the `to` room. A problem PDDL p specifies the initial state of robots, boxes, rooms, and the final goal. For instance, `(at-robby robot2 room3)` means that robot2 is initially at room3. The predicate `(at ball11 room2)` specifies the goal condition that ball11 must eventually be moved to room2. A plan constitutes a sequence of actions to reach the goal. For instance, one action could be `(move robot2 room3 room1)`, moving robot2 from room3 to room1. If robot2 is not already in room3, this action is considered illegal, and the environment will produce an error. For a complete example of domain d , problem p , and plan q , see Listings 1, 2, and 7, respectively in the Appendix.

and will be in the `to` room. A problem PDDL p specifies the initial state of robots, boxes, rooms, and the final goal. For instance, `(at-robby robot2 room3)` means that robot2 is initially at room3. The predicate `(at ball11 room2)` specifies the goal condition that ball11 must eventually be moved to room2. A plan constitutes a sequence of actions to reach the goal. For instance, one action could be `(move robot2 room3 room1)`, moving robot2 from room3 to room1. If robot2 is not already in room3, this action is considered illegal, and the environment will produce an error. For a complete example of domain d , problem p , and plan q , see Listings 1, 2, and 7, respectively in the Appendix.

Large Language Models (LLMs). We assume access to a powerful language model LLM. $\text{LLM}_n(X)$ denotes sampling n responses from the LLM given prompt X . Following the prior works, we set a temperature of $\tau = 0$ for sampling with $n = 1$ (i.e., greedy sampling), and a temperature of $\tau = 0.7$ for $n > 1$ [5]. Whenever possible, we use zero-shot or one-shot chain-of-thought prompts [14, 29] for the LLM to reason before generating a response.

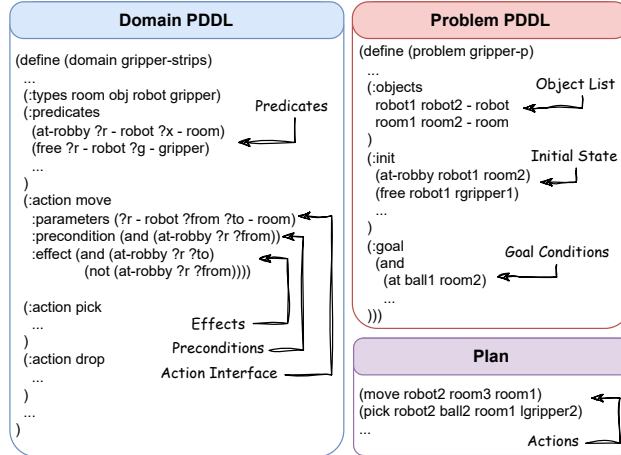


Figure 1: Snippets of PDDL domain, problem, and plan.

4 Method

Given an environment e , its domain NL description and a task NL description, the environment’s object list and action interface, our goal is to model the environment by generating a domain PDDL $\hat{d} \in \mathcal{D}$ and a problem PDDL $\hat{p} \in \mathcal{P}$, such that applying a classical planner C on the PDDL files produces a valid plan for the environment, *i.e.*, $C(\hat{d}, \hat{p})$ is a valid plan for e , *i.e.*, $V_{d,p}(C(\hat{d}, \hat{p})) = 1$.

4.1 Setup

For evaluation, we assume there exists a ground truth domain PDDL $d \in \mathcal{D}$, and a corresponding problem instance $p \in \mathcal{P}$. However, the ground truth is not directly compared to generated $\hat{d}, \hat{p}$, but to validate the plan $\hat{q} := C(\hat{d}, \hat{p})$ by executing the validator of the ground-truth environment, $V_{d,p}(\hat{q})$.

Formally, for each environment e with domain PDDL $d \in \mathcal{D}$, and N tasks with their corresponding ground-truth problem PDDLs $p_{1:N} := (p_1, p_2, \dots, p_N)$, $p_{1:N} \in \mathcal{P}^N$, our goal is to generate a domain PDDL $\hat{d}$, and a sequence of task PDDLs $\hat{p}_{1:N} := (\hat{p}_1, \hat{p}_2, \dots, \hat{p}_N)$ such that the average solve rate $\bar{V}$ is maximized:

$$\operatorname{argmax}_{\hat{d} \in \mathcal{D}, \hat{p}_{1:N} \in \mathcal{P}^N} \bar{V}(\hat{d}, \hat{p}_{1:N}; e) := \frac{1}{N} \sum_{i=1}^N V_{d,p_i}(C(\hat{d}, \hat{p}_i)). \quad (1)$$

Generating accurate $\hat{d}$ and $\hat{p}_{1:N}$ in one attempt is often impractical [9], and some form of feedback is required to refine the response. Guan et al. [9] leverage human expert feedback on $\hat{d}$ to correct the generated domain. However, human feedback may not always be reliable and is not scalable. Before introducing our method that relies on environment feedback instead, we first state our assumptions:

Assumption 1 (Environment access) *We assume the list of objects and action interfaces are known. Furthermore, we assume that executability and verifiability of actions can be observed (through the functions $E_{d,p}$ and $V_{d,p}$).*

Assumption 2 (Natural language description) *We assume the natural language descriptions of the domain and task are both given.*

The action interfaces are equivalent to APIs available to LLM agents. So it is reasonable to assume that the exact API call signatures are known. On the other hand, one may wonder why the object list, which appears in problem PDDLs as illustrated in Figure 1 needs to be assumed to be given, when the NL problem description should describe the objects involved in the planning tasks. This is because the NL description may not refer to the object instances using exactly the same label as the environment induced by d and p . If p refers to a robot as `robot1` but the user specifying the natural language problem description calls it `Jarvis`, then the environment only recognizes `robot1` and not `Jarvis`, so the LLM would have no way to correct this mistake due to trivial name mismatch. See Appendix A.1 for a detailed example of our assumptions on the Grippers environment.

Note that our assumptions do not require the underlying environment to be a PDDL environment, but it can be any environment as long as PDDL is expressive enough to capture the working mechanisms of the environment. For digital agents in virtual environments, the list of objects and action interfaces are just different data objects and APIs available. The assumptions could even hold true for physical agents in the real world, provided recognition and control are sufficiently accurate. In this work, we focus on PDDL environments only, although our framework is more general.

4.2 Difficulty of domain PDDL generation

Generating the correct domain PDDL is challenging, as small mistakes could make the plan search fail. To demonstrate this brittleness, we simulate random omission of k terms, where $0 \leq k \leq 10$, from the action precondition and effects of the original domain d . For instance, in the case of the Grippers (Figure 1), we may create a new synthetic domain by removing the `(at robby ?r ?to)` term from the effects of the move action. Namely, we define $\hat{d}_k \sim \mathbb{P}_k(d)$, where $\mathbb{P}_k(d)$ represents the uniform random removal of k terms. Then, for each generated $\hat{d}_k$, coupled with the ground truth task PDDLs, we compute whether the classical planner is able to find a plan without error and compute the *Plan-Not-Found* rate under k omissions, PNF_k , of the environment.

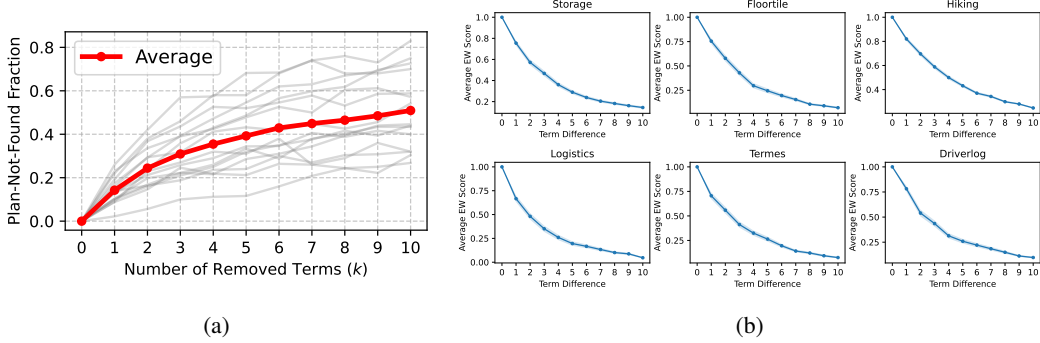


Figure 2: (a) Effect of the number of removed terms on plan search failure. Each gray line shows the PNF_k (Plan-Not-Found) metric for one environment. The red line is the average of all 15 environments. (b) Correlation between average exploration walk (EW) score and average domain difference. The x -axis shows how many terms each pair of domains differs in. The y -axis shows the average EW score over various pairs. All the domains show the average monotonicity of the EW score with respect to term difference.

We empirically measure the value of PNF_k using Monte-Carlo estimation on 15 environments. As shown in Figure 2a, PNF_1 has an average of 0.14 among different environments. This means that on average 14% of the terms in domain PDDLs are so critical that removing them results in a plan-not-found error. This situation is exacerbated for larger k : at $k = 3$, the average PNF_k reaches around 0.3. In practice, the problem PDDL $\hat{p}_i$ also needs to be generated, and the generated domain $\hat{d}$ may have extra terms, both of which may further increase the planning-not-found rate.

4.3 Domain alignment measure via Exploration Walk metrics

Whenever the plan search fails, absolutely no information is available to the LLM about which part of the problem or domain has issues. This is because the underlying search algorithm (such as BFS and A^*) fails and as a result, it does not produce any output. For example, with BFS, it enumerates all paths (possibly several thousand paths or more), and finds none satisfy the goal conditions, leaving the plan search without any useful insights. As an alternative, we introduce the Exploration Walk (EW): a smooth feedback signal that provides incremental feedback for LLM in-context learning. EW both provides a mechanism to gather richer feedback information that feeds into LLM context for PDDL refinement, as well as computing a smooth scoring metric that to compare multiple PDDLs and guide the refinement process forward.

Intuitively, the idea is to take legal random action sequences and verify their executability under LLM’s “mental model” environment induced by an LLM-generated PDDL domain. This is analogous to the *retrodiction* step in scientific methodology, where existing observations and experimental data need to be explained by the existing model.

And in the other direction, EW takes executable random action sequences from an LLM-generated PDDL domain and verifies whether they are correct in the real environment. This is analogous to *hypothesis testing* in scientific methodology, where new predictions are verified experimentally.

We now describe the EW and EW metrics formally. We define an Exploration Walk of length T to be any action sequence sampled from a strictly positive distribution $\mathbb{P}_{d,p,T}$ over executable T -step action sequences in $\mathcal{A}^*$ corresponding to domain d and task p . We assume the probability of non-executable action sequences to be zero under $\mathbb{P}_{d,p,T}$. In other words, $\forall q_{1:T}, \mathbb{P}_{d,p,T}(q_{1:T}) > 0$ iff $E_{d,p}(q_{1:T}) = 1$.

For the rest of this paper, we use the simplest possible EW, with a uniform distribution over valid actions at each step. Note that to sample uniform random EW from the ground truth environment induced by d and p , we do not need direct access to the full d and p . We only need the list of objects in p and the action interface in d , and executability checker $E_{d,p}$, consistent with our Assumption 1. At each step, running $E_{d,p}$ on all possible actions yields the legal actions at that step for EW.

Given an EW distribution, we define an EW metric using the fractions of executability of EW walks from one domain under another, averaged over all different lengths.

Definition 1 (EW Metrics) Let $p_{1:N}$ and $\hat{p}_{1:N}$ be problems in domain d and $\hat{d}$ respectively, such that the set of objects in p_j and $\hat{p}_j$ are consistent. We define the one-sided measure $m_{d \rightarrow \hat{d}}$ and the symmetric one $m_{d \leftrightarrow \hat{d}}$ for the degree of alignment between two domains d and $\hat{d}$ as:

$$\begin{aligned} m_{d \rightarrow \hat{d}}(p_{1:N}, \hat{p}_{1:N}) &:= \frac{1}{NT_{\max}} \sum_{j=1}^N \sum_{T=1}^{T_{\max}} \mathbb{E}_{q \sim \mathbb{P}_{d, p_j, T}} [E_{\hat{d}, \hat{p}_j}(q)] \\ m_{d \leftrightarrow \hat{d}}(p_{1:N}, \hat{p}_{1:N}) &:= 2 / (1/m_{d \rightarrow \hat{d}}(p_{1:N}, \hat{p}_{1:N}) + 1/m_{\hat{d} \rightarrow d}(\hat{p}_{1:N}, p_{1:N})), \end{aligned} \quad (2)$$

where $T_{\max}$ is the largest EW walk length.

$m_{d \rightarrow \hat{d}}$ measures what fraction of EWs sampled from domain d are executable on the domain $\hat{d}$. Then, $m_{d \leftrightarrow \hat{d}}$ takes the harmonic mean of $m_{d \rightarrow \hat{d}}$ and $m_{\hat{d} \rightarrow d}$ to produce the final EW measure. This metric has two favourable properties: (1) it ensures that $m_{d \rightarrow \hat{d}} = m_{\hat{d} \rightarrow d}$, thereby providing a consistent measure of similarity regardless of the order of domain comparison. (2) the harmonic mean is resistant to trivial domain similarity inflation. By employing the harmonic mean rather than the arithmetic mean, the symmetric EW metric prevents domains that are overly permissive (e.g., domains where all actions are permissible without any preconditions) from being similar to more restrictive domains. For example, in a scenario where domain $\hat{d}$ allows all possible actions without restrictions, $m_{d \rightarrow \hat{d}} = 1$. An arithmetic mean in this context would yield $m_{d \leftrightarrow \hat{d}} \geq 0.5$, overestimating the similarity. In contrast, the harmonic mean results in $m_{d \leftrightarrow \hat{d}} = \epsilon$, where ($\epsilon \ll 1$) for most cases.

Note that while the PDDL problems $p_{1:N}$ and $\hat{p}_{1:N}$ appear in the definition of EW metrics, we only use the fact there are aligned object sets in them. We could also use an arbitrarily sampled object list to form an $\tilde{P}$ and pair $\tilde{P}$ with D and $\hat{D}$ for EW metrics. But since for PDDL generation, we already generate $\hat{p}_{1:N}$, it is more convenient to use them.

Importantly, EW metrics can be computed without direct access to the full ground truth domain d and problems p 's. As established before, to sample uniform random EW, we just need access to the object list and action interface, plus the environment executability checker of the source domain. So even for $m_{d \rightarrow \hat{d}}$, where the EW action sequences come from d , we do not need more than what is available through Assumption 1.

To demonstrate the relationship between $m_{d \leftrightarrow \hat{d}}$ and domain disparity, we use the same simulated random omission study setup from Sec. 4.2. For a pair of modified domains, we count the number of terms that differ, and inspect $m_{d \leftrightarrow \hat{d}}$ as function of increasing number of differing terms in Figure 2b for six example domains (see Figure 4 in the Appendix for the full set). We observe that, on average, a greater discrepancy in the number of terms between two domains correlates with a reduced EW score $m_{d \leftrightarrow \hat{d}}$. This observation provides additional support to the use of the EW score as an effective measure for domain differences.

4.4 Leveraging LLMs to generate PDDL files

We now show our overall LLM-based method for PDDL generation using the EW score to guide and measure the progress of domain generation. To illustrate the process, we first focus on a domain d with a single task p . Recall that we are given NL description of the environment domain d_{NL} and problem p_{NL} (Assumption 2), as well as the object list in p and action interface from d (Assumption 1). Then, by using d_{NL} , p_{NL} , and access to environment action feedback, we seek to generate $\hat{d} \in \mathcal{D}$, $\hat{p} \in \mathcal{P}$.

Our method starts by initializing templated $\hat{d}^{(0)}$ based on action interfaces and templated $\hat{p}^{(0)}$ using object list. Example template $\hat{d}^{(0)}$ and $\hat{p}^{(0)}$ are shown in Listings 6 and 4 of Appendix A.1. We then use an LLM to improve the initial $\hat{d}^{(0)}$ and $\hat{p}^{(0)}$.

Given that domain PDDL files are typically more complex than problem PDDL files, our strategy prioritizes the generation of a problem PDDL file $\hat{p}$ first, followed by the domain $\hat{d}$. This approach enables us to assess the quality of the generated domain immediately. Moreover, prior works on code generation [4], tree-of-thought [31], and self-debug [5] have found that taking multiple samples from the LLM response and taking the best response leads to better performance. However, they often require an evaluation metric on the generated response (such as unit test cases, or execution traces). Here, we use the EW metric introduced in Section 4.3 to serve as an evaluator of the generated domain. These considerations lead to our proposed Algorithm 1. We emphasize again that the

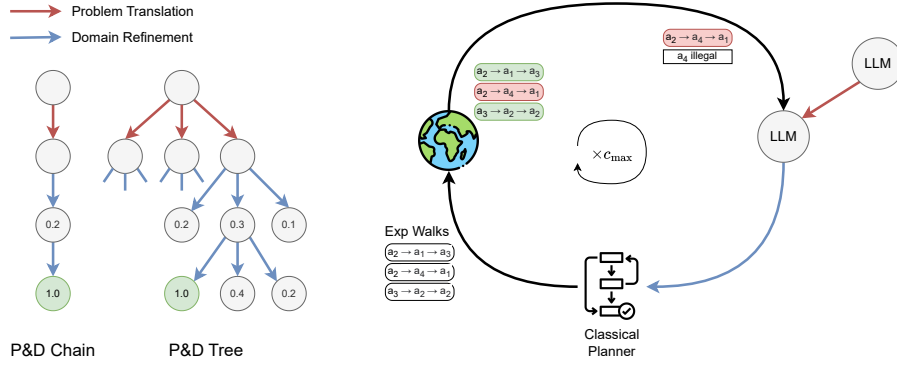


Figure 3: Overview of our method. *Right*: The process begins with natural language descriptions translated into problem PDDL by the LLM (red arrows). Then a domain is generated and refined through iterative cycles involving exploration walks in the environment, interaction with a classical planner, and feedback from the LLM (blue/black arrows). *Left*: The iterative refinement process depicted on the right corresponds to single paths in the structures shown on the left. Each node represents a state in the refinement process, with arrows indicating problem translation (red), domain refinement (blue).

ground-truth domain and problem d, p are only used to take exploration walks and evaluate a plan through the environment in 1.

Algorithm 1 Generating Domain PDDL and Problem PDDL Using Environment Feedback

Require: Natural language descriptions $d_{\text{NL}}, p_{\text{NL}}$, environment action interface.

- 1: $\hat{p}^{(1)}, \hat{p}^{(2)}, \dots, \hat{p}^{(n_p)} \leftarrow \text{LLM}_{n_p}(p_{\text{NL}})$ {Problem PDDL candidates}
- 2: **for** $i = 1, 2, \dots, n_p$ **do**
- 3: $h^{(i)} \leftarrow [\hat{p}^{(i)}, d_{\text{NL}}]$ {Keep a history of conversation}.
- 4: $\hat{d}_{\text{best}}^{(i)} \leftarrow d_{\text{NL}}$ {Initialize with an empty template}.
- 5: **for** $c = 1, 2, \dots, c_{\text{max}}$ **do**
- 6: $\hat{d}^{(i,1)}, \hat{d}^{(i,2)}, \dots, \hat{d}^{(i,n_d)} \leftarrow \text{LLM}_{n_d}(h^{(i)})$
- 7: $\hat{d}^{(c)} \leftarrow \text{argmax}_{\hat{d} \in \{\hat{d}^{(i,1)}, \dots, \hat{d}^{(i,n_d)}\}} m_{d \leftrightarrow \hat{d}}(p, \hat{p}^{(i)})$ {Evaluate LLM responses using EW}
- 8: $f^{(c)} \leftarrow$ Natural language feedback from EW on d, p .
- 9: $h^{(i)} \leftarrow h^{(i)} + [\hat{d}^{(c)}, f^{(c)}]$
- 10: $\hat{d}_{\text{best}}^{(i)} \leftarrow \text{argmax}_{\hat{d} \in \{\hat{d}^{(c)}, \hat{d}_{\text{best}}^{(i)}\}} m_{d \leftrightarrow \hat{d}}(p, \hat{p}^{(i)})$
- 11: **end for**
- 12: **end for**
- 13: $\hat{d}, \hat{p} \leftarrow \text{argmax}_{\{\hat{d}_{\text{best}}^{(i)}, \hat{p}^{(i)}\} | i=1,2,\dots,n_p} m_{d \leftrightarrow \hat{d}_{\text{best}}^{(i)}}(p, \hat{p}^{(i)})$
- 14: **return** $\hat{d}, \hat{p}$ {Return the final refined domain and problem PDDLs}

Note that each environment contains $N > 1$ problems, therefore, we need to translate all problem instances into PDDL. Similar to Liu et al. [15], given one problem $p_{1_{\text{NL}}}$ and its generated translation $\hat{p}_1$, we translate the rest of the problems $p_{2:N_{\text{NL}}}$ in a one-shot manner. That is, we generate $\hat{p}_i := \text{LLM}_1(p_{1_{\text{NL}}}, \hat{p}_1, p_{i_{\text{NL}}})$ as the final problem translation for problem i for all $2 \leq i \leq N$.

5 Experiments

Dataset. We consider PDDL files from real environments, taking nine domains from a combination of domain PDDLs from Liu et al. [15] and Seipp et al. [22]. The LLM may have seen the mentioned domains in its pre-training data, which is a common issue for current benchmarks. To mitigate this issue, we also modify the original Grippers domain, and create a modified domain called “Grippersood” domain, to ensure no LLM has seen it previously. We generate natural domain descriptions for

all PDDL files by back-translating them using GPT-4 and manually inspecting and modifying the translations for correctness. For each environment, we consider one domain PDDL d and $N = 10$ problem PDDLs $p_{1:N}$. We use one problem for domain translation and EW evaluation, and all problems for evaluating a final domain response. We reserve the Blocksworld environment as an in-context example for prompting the LLM. As such, we do not evaluate the Blocksworld environment itself in our evaluations. See Appendices A.1 and C for more details on dataset curation.

Feedback Format. The natural language feedback given to LLM is in the following form: [Action sequence] [State description]. That is, we first provide LLM with the sequence of actions taken from one exploration walk, up until one action fails. Then, we provide the environment state description from the last step. We show an example of environment feedback and LLM response for the Termes environment in Listings 9 in the Appendix. We deliberately choose a simple feedback format to maintain the general applicability of our framework.

Baselines and Metrics. We use GPT-4 [1] (gpt-4-1106-preview) as the LLM since models with lower capability may struggle with syntax errors [9]. We consider the following methods: (1, 2) **Intrinsic Planning (CoT)**: where the language model generates a complete plan without the help of any external planning library, based on the given descriptions, both with and without chain-of-thought prompting. This baseline does not leverage any classical planner or PDDL translation. (3) **P&D Chain**: Our proposed method (Algorithm 1) with $n_d = n_p = 1$. (4) **P&D Tree**: Our proposed method with multiple response generations ($n_d = 10, n_p = 5$). (5) **P&D Tree + DomProp**: Our proposed method with multiple response generations and domain proposals for each problem (see Appendix B.2). Following prior works [17, 5], we set a maximum conversation turns of $c_{\max} = 4$.

We run each algorithm for four seeds and compute the Best@4 metric, which takes the highest score among the four seeds. We report two metrics: (1) tasks solved², measuring the fraction of the $N = 10$ tasks successfully solved (Eq. (1)), and (2) EW score, comparing the final domain through running exploration walks on all N problems (Eq. (2) with $T_{\max} = 10$). We use the original fast-downward [11] library for planning, the modified fast-downward library from text-world [6] for python-compatible state explorations, and the VAL [12] library to validate plans.

Results. Table 2 shows the final results on various environments. We consider a domain generation to be solved if a method achieves > 0.5 solve rate since we observe the rest of the errors are problem translation errors rather than domain translation errors. Our proposed method solves 7 out of 10 domains, compared to 3 solved by the Intrinsic CoT baseline. We also generally observe the correlation of EW score with task solve rate. Particularly, even when the task solve rate is zero, the EW metric shows signs of progress, *e.g.*, in domains such as Barman and Childsnack where all task solve rates are zero, the EW metric shows a clear distinction between method performances. Moreover, when the EW metric is high, such as 1.0, we observe a generated PDDL domain to be very close to the ground-truth domain, and differing in very few predicates. For instance, in the case of the “Hiking” environment, the P&D Chain achieves zero solve rate, but a perfect EW score, which we observe perfect solution in the case of P&D Tree.

Computational Cost. For the results in Table 2 using the GPT-4 model, we used 12.40 million input tokens and 8.73 million output tokens. Computing the EW is relatively negligible compared to the cost of LLM inference. In our experiments, computing the EW score for a single domain-problem pair takes less than two minutes on a 64-core server CPU.

6 Conclusion

In this work, we present a novel approach for modeling planning environments via PDDL translation using large language models (LLMs) and environment feedback, without relying on human intervention. The key contributions include introducing the Exploration Walk (EW) metric to measure domain similarity and guide domain refinement, and an iterative method that leverages LLMs to generate and refine PDDL domain and problem files. Evaluation on 10 real-world PDDL domains demonstrates the effectiveness of the proposed approach, outperforming a baseline that generates PDDL files in a single attempt without refinement. The method solves 7 out of 10 environments, achieving an average task solve rate of 66% and an average EW score of 0.84.

²Note that a perfect task solve rate does not guarantee exact domain equivalency of the generated domain to the ground truth domain.

Table 2: Best@4 (Tasks solved / Exploration Walk) for different domains. For intrinsic planning no domain is generated, therefore the EW score is not defined.

	Intrinsic No CoT	Intrinsic CoT	P&D Chain ($n_d = 1, n_p = 1$)	P&D Tree ($n_d = 10, n_p = 5$)	P&D Tree + DomProp ($n_d = 10, n_p = 5$)
Barman	0.00 / –	0.00 / –	0.00 / 0.93	0.00 / 1.00	0.00 / 1.00
Childsnack	0.00 / –	0.00 / –	0.00 / 0.57	0.00 / 1.00	0.00 / 1.00
Driverlog	0.00 / –	0.00 / –	0.00 / 0.05	0.00 / 0.05	0.00 / 0.60
Floortile	0.00 / –	0.00 / –	0.00 / 0.07	0.90 / 0.94	0.00 / 0.07
Grippers	0.40 / –	0.60 / –	0.10 / 0.39	1.00 / 1.00	1.00 / 1.00
Grippers-ood	0.30 / –	0.30 / –	0.30 / 0.35	0.70 / 0.72	1.00 / 1.00
Hiking	0.00 / –	0.00 / –	0.00 / 1.00	1.00 / 1.00	1.00 / 1.00
Miconic	0.90 / –	1.00 / –	1.00 / 0.84	1.00 / 0.85	1.00 / 1.00
Movie	1.00 / –	1.00 / –	1.00 / 0.07	1.00 / 0.85	1.00 / 0.86
Termes	0.00 / –	0.00 / –	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00
Average	0.26 / –	0.29 / –	0.34 / 0.53	0.66 / 0.84	0.60 / 0.85

The current limitations include potentially insufficient and efficient exploration caused by random EW. More sophisticated EW strategies could improve the success rate while lowering the cost in the future. For example, strategies from the reinforcement learning literature (*e.g.*, [27, 21]) could be adapted to improve exploration efficiency and success rates. Another limitation is that we have only applied the framework to PDDL environments, despite it being applicable to digital or even physical environments. We hope this work will inspire further research at the intersection of language models and planning, enabling the development of more advanced and autonomous planning systems.

References

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] A. Ahmetoglu, M. Y. Seker, J. Piater, E. Oztup, and E. Ugur. Deepsym: Deep symbol generation and rule learning for planning from unsupervised robot interaction. *Journal of Artificial Intelligence Research*, 75:709–745, 2022.
- [3] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.
- [4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=KuPixIqPiq>.
- [6] M.-A. Côté, A. Kádár, X. Yuan, B. Kybartas, T. Barnes, E. Fine, J. Moore, R. Y. Tao, M. Hausknecht, L. E. Asri, M. Adada, W. Tay, and A. Trischler. Textworld: A learning environment for text-based games. *CoRR*, abs/1806.11532, 2018.
- [7] G. Dagan, F. Keller, and A. Lascarides. Dynamic planning with a llm. *arXiv preprint arXiv:2308.06391*, 2023.
- [8] N. Dziri, X. Lu, M. Sclar, X. L. Li, L. Jiang, B. Y. Lin, S. Welleck, P. West, C. Bhagavatula, R. Le Bras, et al. Faith and fate: Limits of transformers on compositionality. *Advances in Neural Information Processing Systems*, 36, 2024.
- [9] L. Guan, K. Valmeekam, S. Sreedharan, and S. Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=zDbsSscmuj>.

- [10] M. Han, Y. Zhu, S.-C. Zhu, Y. N. Wu, and Y. Zhu. Interpret: Interactive predicate learning from language feedback for generalizable task planning. In *Robotics: Science and Systems (RSS)*, 2024.
- [11] M. Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.
- [12] R. Howey, D. Long, and M. Fox. Val: Automatic plan validation, continuous effects and mixed initiative planning using pddl. In *Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence, ICTAI '04*, page 294–301, USA, 2004. IEEE Computer Society. ISBN 076952236X. doi: 10.1109/ICTAI.2004.120. URL <https://doi.org/10.1109/ICTAI.2004.120>.
- [13] IPC. International planning competition, 1998. URL <https://www.icaps-conference.org/competitions/>.
- [14] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213, 2022.
- [15] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. Llm+p: Empowering large language models with optimal planning proficiency. *ArXiv*, abs/2304.11477, 2023. URL <https://api.semanticscholar.org/CorpusID:258298051>.
- [16] Q. Lyu, S. Havaldar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidianaki, and C. Callison-Burch. Faithful chain-of-thought reasoning. In J. C. Park, Y. Arase, B. Hu, W. Lu, D. Wijaya, A. Purwarianti, and A. A. Krisnadhi, editors, *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 305–329, Nusa Dua, Bali, Nov. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.ijcnlp-main.20. URL <https://aclanthology.org/2023.ijcnlp-main.20>.
- [17] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. Self-refine: Iterative refinement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=S37h0erQLB>.
- [18] D. McDermott. The 1998 ai planning systems competition. *AI Magazine*, 21(2):35–55, 2000.
- [19] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. I. Wang, and X. V. Lin. Lever: Learning to verify language-to-code generation with execution. In *Proceedings of the 40th International Conference on Machine Learning (ICML'23)*, 2023.
- [20] J. Oswald, K. Srinivas, H. Kokel, J. Lee, M. Katz, and S. Sohrabi. Large language models as planning domain generators. In *34th International Conference on Automated Planning and Scheduling*, 2024. URL <https://openreview.net/forum?id=C88wQIv0aJ>.
- [21] S. Sagar, A. Taparia, and R. Senanayake. Failures are fated, but can be faded: Characterizing and mitigating unwanted behaviors in large-scale vision and language models. *arXiv preprint arXiv:2406.07145*, 2024.
- [22] J. Seipp, Á. Torralba, and J. Hoffmann. PDDL generators. <https://doi.org/10.5281/zenodo.6382173>, 2022.
- [23] T. Silver, V. Hariprasad, R. S. Shuttlesworth, N. Kumar, T. Lozano-Pérez, and L. P. Kaelbling. PDDL planning with pretrained large language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022. URL <https://openreview.net/forum?id=1QMMUB4zf1>.
- [24] T. Silver, R. Chitnis, N. Kumar, W. McClinton, T. Lozano-Pérez, L. Kaelbling, and J. B. Tenenbaum. Predicate invention for bilevel planning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 12120–12129, 2023.
- [25] T. Silver, S. Dan, K. Srinivas, J. Tenenbaum, L. Kaelbling, and M. Katz. Generalized planning in PDDL domains with pretrained large language models. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2024.
- [26] K. Stein and A. Koller. Autoplanbench:: Automatically generating benchmarks for llm planners from pddl. *arXiv preprint arXiv:2311.09830*, 2023.

- [27] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018. ISBN 0262039249.
- [28] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. URL <https://openreview.net/forum?id=YXog14uQU0>.
- [29] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [30] Y. Xie, C. Yu, T. Zhu, J. Bai, Z. Gong, and H. Soh. Translating natural language to planning goals with large-language models. *arXiv preprint arXiv:2302.05128*, 2023.
- [31] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. R. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=5Xc1ecx01h>.
- [32] T. Zhang, T. Yu, T. Hashimoto, M. Lewis, W.-t. Yih, D. Fried, and S. Wang. Coder reviewer reranking for code generation. In *International Conference on Machine Learning*, pages 41832–41846. PMLR, 2023.

Table 3: Summary of Notation and Definitions

Notation	Description
$1 : N$	Sequence of integers ranging from 1 to N
$\mathcal{A}^*$	Set comprising all possible sequences of elements drawn from set $\mathcal{A}$
$2^{\mathcal{A}}$	Power set of $\mathcal{A}$
$\mathcal{D}$	Set of all possible domains in PDDL
$\mathcal{P}$	Set of all possible problems in PDDL
$\mathcal{A}$	Set of all possible actions in PDDL
$\perp$	Planning error
$C : \mathcal{D} \times \mathcal{P} \rightarrow \mathcal{A}^* \cup \{\perp\}$	Classical planner function that takes a domain $d \in \mathcal{D}$ and a problem $p \in \mathcal{P}$ and produces a plan q
$V_{d,p}(q) : \mathcal{A}^* \rightarrow \{0, 1\}$	Plan validator function for domain d and problem p , returns 1 if plan q is valid, otherwise 0
$E_{d,p} : \mathcal{A}^* \rightarrow \{0, 1\}$	Plan execution checker for domain d and problem p , returns 1 if action sequence is executable, otherwise 0
$\mathcal{S}$	Set of all possible states
$A_{d,p} : \mathcal{S} \rightarrow 2^{\mathcal{A}}$	Function delineating the set of legal actions given the current state for domain d and problem p
$S_{d,p} : \mathcal{A} \times \mathcal{S} \rightarrow \mathcal{S}$	State transition function, determines the next state given the current state and action in domain d and problem p
$s_{d,p,0}$	Initial state induced by domain d and problem p
$\text{LLM}_n(X)$	Sampling n responses from the LLM given prompt X

A Dataset

A.1 Dataset Details.

Dataset Examples. We provide an example of each file for the Grippers environment: (1) The ground-truth domain d (Listing 1) of ground truth PDDL domain (2) One ground-truth problem p (Listing 2) (3) Domain natural language description along with a PDDL template for action interfaces d_{NL} (Listings 5 and 6) (4) Problem natural language description along with a PDDL template with the list of objects (Listings 3 and 4)

```

1 (define (domain gripper-strips)
2   (:requirements :strips :typing)
3   (:types room obj robot gripper)
4   (:predicates (at-robby ?r - robot ?x - room)
5                 (at ?o - obj ?x - room)
6                 (free ?r - robot ?g - gripper)
7                 (carry ?r - robot ?o - obj ?g - gripper))
8
9   (:action move
10    :parameters (?r - robot ?from ?to - room)
11    :precondition (and (at-robby ?r ?from))
12    :effect (and (at-robby ?r ?to)
13               (not (at-robby ?r ?from))))
14
15  (:action pick

```

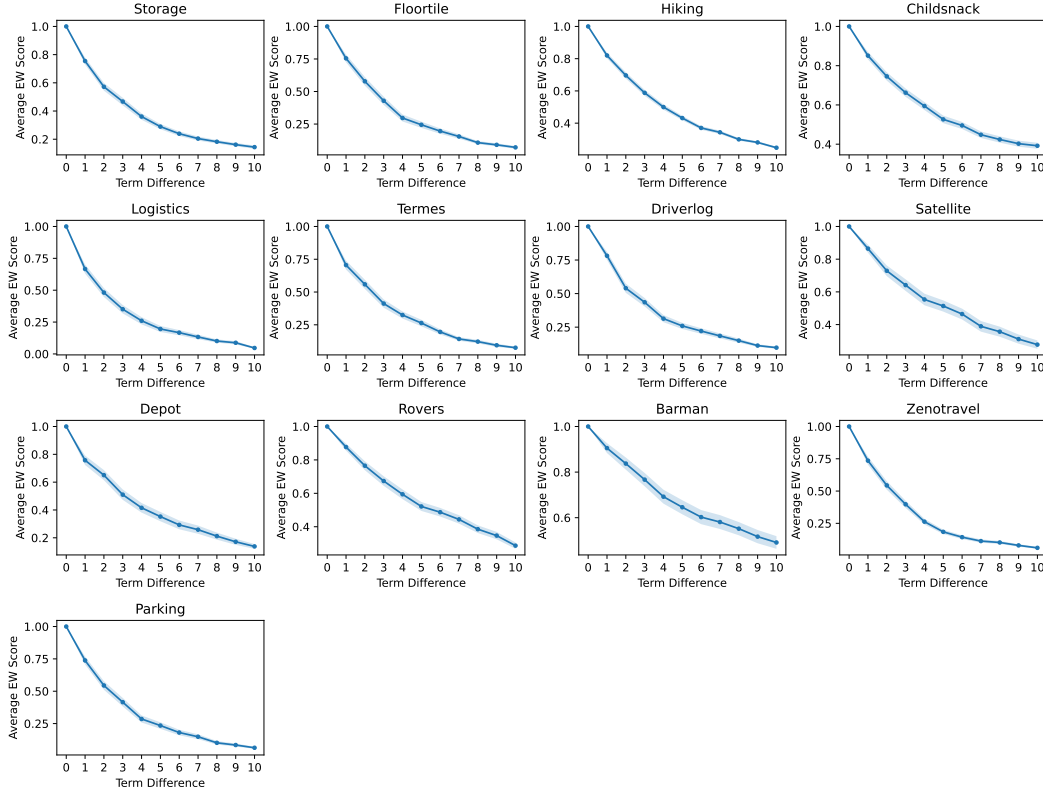


Figure 4: Correlation between average exploration walk score and average domain difference

```

16 :parameters (?r - robot ?obj - obj ?room - room ?g -
    gripper)
17 :precondition (and (at ?obj ?room) (at-robby ?r ?room) (
    free ?r ?g))
18 :effect (and (carry ?r ?obj ?g)
19             (not (at ?obj ?room))
20             (not (free ?r ?g))))
21
22 (:action drop
23   :parameters (?r - robot ?obj - obj ?room - room ?g -
24               gripper)
25   :precondition (and (carry ?r ?obj ?g) (at-robby ?r ?room))
26   :effect (and (at ?obj ?room)
27               (free ?r ?g)
28               (not (carry ?r ?obj ?g))))

```

Listing 1: Grippers domain PDDL [15].

```

1 (define (problem gripper-2-3-4)
2   (:domain gripper-strips)
3   (:objects robot1 robot2 - robot
4             rgripper1 lgripper1 rgripper2 lgripper2 - gripper
5             room1 room2 room3 - room
6             ball1 ball2 ball3 ball4 - obj)
7   (:init
8     (at-robby robot1 room2)

```

```

9      (free robot1 rgripper1)
10     (free robot1 lgripper1)
11     (at-robby robot2 room3)
12     (free robot2 rgripper2)
13     (free robot2 lgripper2)
14     (at ball1 room3)
15     (at ball2 room1)
16     (at ball3 room1)
17     (at ball4 room3)
18   )
19   (:goal
20     (and
21       (at ball1 room2)
22       (at ball2 room2)
23       (at ball3 room3)
24       (at ball4 room3)
25     )
26   )
27 )

```

Listing 2: Grippers problem PDDL.

```

1  You control two robots, each equipped with a left and right
   gripper, capable of moving objects (balls) between
   different rooms.
2
3  Initially:
4  - Robot1 is in room2 and both its grippers (rgripper1 and
   lgripper1) are free.
5  - Robot2 is in room3 and both its grippers (rgripper2 and
   lgripper2) are free.
6  - Ball1 and Ball4 are in room3.
7  - Ball2 and Ball3 are in room1.
8
9  Your goal is to achieve the following configuration:
10 - Ball1 must be moved to room2.
11 - Ball2 must be moved to room2.
12 - Ball3 must remain in room3.
13 - Ball4 must remain in room3.

```

Listing 3: Grippers problem natural language.

```

1  (define (problem gripper-2-3-4)
2    (:domain gripper-strips)
3    (:objects lgripper1 lgripper2 rgripper1 rgripper2 - gripper
              ball1 ball2 ball3 ball4 - obj robot1 robot2 - robot
              room1 room2 room3 - room)
4    (:init )
5    (:goal (and ))
6  )

```

Listing 4: Grippers problem template PDDL.

```

1  The gripper domain involves a world with multiple rooms,
   robots, and objects (balls). Each robot has two grippers

```

```

2   that can be used to pick up and drop objects. The goal is
3   to move objects from their initial locations to the
4   desired goal locations using the robots and their grippers.
5
6   The domain includes three actions:
7
8   1. move: This action allows a robot to move from one room to
9   another. The precondition is that the robot must be in the
10  starting room. The effect is that the robot is no longer
11  in the starting room and is now in the destination room.
12
13  2. pick: This action allows a robot to pick up an object using
14  one of its grippers. The preconditions are that the object
15  and the robot must be in the same room, and the specified
16  gripper must be free (not holding any object). The effect
17  is that the robot is now carrying the object with the
18  specified gripper, the object is no longer in the room,
19  and the gripper is no longer free.
20
21  3. drop: This action allows a robot to drop an object it is
22  carrying in a specific room using one of its grippers. The
23  preconditions are that the robot must be carrying the
24  object with the specified gripper and the robot must be in
25  the specified room. The effect is that the object is now
26  in the room, the gripper is free, and the robot is no
27  longer carrying the object with that gripper.

```

Listing 5: Grippers domain natural language.

```

1  (define (domain gripper-strips)
2    (:requirements :strips :typing)
3    (:types room obj robot gripper)
4    (:predicates)
5
6    (:action move
7      :parameters (?r - robot ?from ?to - room)
8      :precondition ()
9      :effect ())
10
11    (:action pick
12      :parameters (?r - robot ?o - obj ?room - room ?g - gripper)
13      :precondition ()
14      :effect ())
15
16    (:action drop
17      :parameters (?r - robot ?o - obj ?room - room ?g - gripper)
18      :precondition ()
19      :effect ()))

```

Listing 6: Grippers domain PDDL template.

```

1  (move robot2 room3 room1)
2  (pick robot2 ball2 room1 lgripper2)
3  (move robot2 room1 room2)
4  (drop robot2 ball2 room2 lgripper2)

```

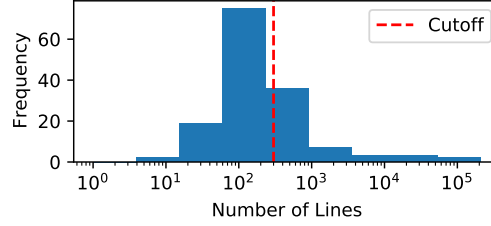


Figure 5: Histogram of average number of lines of domains in [22].

```

5 (move robot1 room2 room1)
6 (pick robot1 ball3 room1 lgripper1)
7 (move robot1 room1 room3)
8 (pick robot1 ball1 room3 rgripper1)
9 (drop robot1 ball3 room3 lgripper1)
10 (move robot1 room3 room2)
11 (drop robot1 ball1 room2 rgripper1)

```

Listing 7: Grippers problem plan example.

A.2 Criticality of predicate design.

Here, we give an example on the delicacy of predicate design. Consider the Grippers environment, where each robot has two grippers: left gripper and right gripper. In our experiments, one of the main predicates that the LLM incorrectly generates is the `free` predicate (see Listing 8). This predicate keeps track of whether a gripper is free or not. Therefore, at first sight, `(free ?g - gripper)` seems a natural choice to show a particular gripper is not occupied and hence is capable of picking a ball. However, when designed this way, in contrast to `(free ?r - robot ?g - gripper)` (missing the robot argument), this small detail causes the final domain to be entirely wrong! The reason is that there would no longer be any association between a robot and its two grippers. Therefore, on the incorrect domain, one robot will be able to pickup an object with the gripper of another robot! In fact, we observe that this incorrect design for the `free` predicate, is the reason behind the failure of the “P&D Chain” method in Table 2.

We provide one more example from the Barman environment, illustrating the criticality of predicate design. The Barman environment involves actions related to manipulating containers (e.g., shot glasses, shakers) to prepare and serve drinks using various ingredients. One of the key predicates used in the domain is `(used ?c - container ?b - beverage)`, which keeps track of which beverage has been used in a specific container. This is important for actions like refilling or cleaning, where knowing the specific beverage type is essential to ensure conformation to the environment rules (e.g., a container can be refilled only with the beverage that it already had, otherwise, it needs to be cleaned first). However, we have observed that when the LLM generates the domain, it sometimes mistakenly omits the beverage argument, simplifying the predicate to `(used ?c - container)`. At first glance, this might seem like a harmless simplification, as the container usage is still tracked. However, this change results in significant problems in the overall domain behavior. Since the beverage is no longer specified, the domain can no longer differentiate between containers used for different types of beverages. This leads to situations where a container that has already been used for one beverage can be incorrectly treated as if it can hold another beverage without requiring proper cleaning or resetting actions. Such a mistake can cause the final domain to generate invalid plans, as the planner will fail to ensure that containers are used properly with respect to their contents, leading to cascading errors in tasks like mixing drinks, cleaning containers, or pouring from shakers.

```

1 (define (problem gripper-2-3-4)
2   (:domain gripper-strips)
3   (:objects robot1 robot2 - robot
4           rgripper1 lgripper1 rgripper2 lgripper2 - gripper

```

```

5         room1 room2 room3 - room
6         ball1 ball2 ball3 ball4 - obj)
7     (:init
8         (at-robby robot1 room2)
9         (free rgripper1) ; Correct: (free robot1 rgripper1)
10        (free lgripper1) ; Correct: (free robot1 lgripper1)
11        (at-robby robot2 room3)
12        (free rgripper2) ; Correct: (free robot2 rgripper2)
13        (free lgripper2) ; Correct: (free robot2 lgripper2)
14        (at ball1 room3)
15        (at ball2 room1)
16        (at ball3 room1)
17        (at ball4 room3)
18    )
19    (:goal
20        (and
21            (at ball1 room2)
22            (at ball2 room2)
23            (at ball3 room3)
24            (at ball4 room3)
25        )
26    )
27 )

```

Listing 8: Incorrect generated grippers problem PDDL. The free predicate has only one parameter.

B Implementation Details

In this section, we explain our implementation details.

B.1 One-shot prompting

To generate PDDL files (problem PDDL and domain PDDL), we always include a one-shot example prompt from the BlocksWorld environment. This environment is concise easy enough to fit into context, and explanatory enough to demonstrate example to the LLM for better output steerability. This includes problem generation, domain proposal, and problem refinement. For instance, when prompting the LLM to generate problem translation from natural language, *e.g.*, $\text{LLM}(p_{\text{NL}})$, we also prompt the LLM with an example from Blocksworld.

B.2 P&D Tree with Domain Proposal

As discussed in A.2, predicate design is challenging. Therefore, in one variant of our method, which we call “P&D Tree DomProp”, we propose for the LLM to first draft a domain proposal, then generate a problem PDDL based on the predicates found in the draft. This way, the LLM first generates domain-aware predicates, then generates the problem PDDL. Formally, line one in Algorithm 1 will be changed to the following two lines:

$$\begin{aligned} \hat{d}_{\text{pr}}^{(1)}, \hat{d}_{\text{pr}}^{(2)}, \dots, \hat{d}_{\text{pr}}^{(n_p)} &\leftarrow \text{LLM}_{n_p}(d_{\text{NL}}) \\ \hat{p}^{(i)} &\leftarrow \text{LLM}_1(\hat{d}_{\text{pr}}^{(i)}, p_{\text{NL}}) \text{ for all } 1 \leq i \leq n_p \end{aligned}$$

where the problem PDDL is generated by first creating a domain proposal.

B.3 Domain Refinement Strategy

Refinement Interface. For the domain refinement stage, in our early experiments we observed that prompting the LLM to regenerate the domain results into redundant output generation and

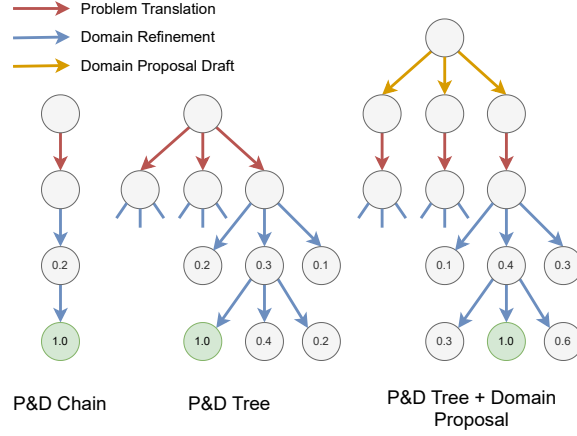


Figure 6: Overview of our method with domain proposal. To generate a problem PDDL, the LLM first drafts a domain proposal to find suitable predicates for the problem PDDL. Then, the draft is discarded, and the domain refinement stage starts.

more importantly, sometimes modifies incorrect parts of the domain. For instance, the LLM had a high tendency towards changing the action interface signature, despite the instructions explicitly mentioning not to change the signature. As such, we provide a python interface for the LLM to modify a domain. The interface provides the LLM with the following two functions:

```
add_or_update_predicates(
    predicates: List[str]
)
modify_action(
    action_name: str,
    new_preconditions: List[str],
    new_effects: List[str]
)
```

The first function adds predicates to the list of already created predicates, and the second one modifies the preconditions and effects of a particular action. Guan et al. [9] use a similar approach where they generate the domain PDDL one action at a time, and gradually create predicates. However, our python function interface allows for more flexibility, such as more convenient implementation as well as enabling the LLM to modify an action several times, or introduce predicates in between reasoning steps.

Domain Rating. Our main domain rating originates from the EW metric. When generating domain refinement strategies, the LLM may make mistakes hence failing before even the EW metric could be computed. For instance, the modification may be invalid, containing syntax error, or failing to fill parts of the template. To facilitate incorporating these into the EW metric strategy, we create the following rating system for each domain refinement modification:

Description	Rating
Exploration Walk Executable	$0 \leq \text{EW Score} \leq 1$
No initial action possible	-1
Invalid domain modification (<i>e.g.</i> , undefined predicates)	-2
Domain sanity check failure (<i>e.g.</i> , empty effect list)	-3
Invalid domain modification	-4
No domain modification	-5

when the EW metric is perfect (*i.e.*, equals 1.0), we also run the planning on the environment (*i.e.*, evaluate $V_{d,p}(\cdot)$) and stop early if the plan is valid.

```
1 User:
2 Incorrect. The environment returned the following error:
3
4 Sampled a set of consecutive random actions from the ground
  truth environment, but the actions are not executable in
  the generated environment.
5 Executing the following actions sequentially on the
  environment:
6 (move pos-2-0 pos-3-0 n0)
7 (move pos-3-0 pos-2-0 n0)
8 (create-block pos-2-0)
9 (move pos-2-0 pos-3-0 n0)
10 (move pos-3-0 pos-2-0 n0)
11 (place-block pos-2-0 pos-1-0 n0 n1)
12 (remove-block pos-2-0 pos-1-0 n1 n0)
13 (place-block pos-2-0 pos-2-1 n0 n1)
14 (remove-block pos-2-0 pos-2-1 n1 n0)
15 (place-block pos-2-0 pos-3-0 n0 n1)
16 (create-block pos-2-0)
17 (place-block pos-2-0 pos-2-1 n0 n1)
18 (create-block pos-2-0)
19 (place-block pos-2-0 pos-1-0 n0 n1)
20 (create-block pos-2-0)
21 (move-up pos-2-0 n0 pos-1-0 n1)
22
23 Result: Error when executing the action (move-up pos-2-0 n0
  pos-1-0 n1).
24 Current state: The robot is at position pos-2-0.
25 This action is not executable on the environment.
26
27 Please reason about the issue with your generated code. The
  current domain pddl is as follows:
28
29 ‘‘‘pddl
30 [Current Generated PDDL]
31 ‘‘‘
32
33 In your response, please generate a new code to fix the issue.
34
35 Assistant:
36 It seems that there is an issue with the ‘move-up’ action
  precondition. The precondition for ‘move-up’ should check
  that the destination position ‘?hto’ is one less than the
  starting position ‘?hfrom’, not the other way around. This
  is because the robot is moving to a higher position, so
  the destination height should be greater than the starting
  height.
37
38 Similarly, the precondition for ‘move-down’ should check that
  the destination position ‘?hto’ is one less than the
  starting position ‘?hfrom’, indicating that the robot is
  moving to a lower position.
```

```

39
40 Let's correct the preconditions for 'move-up' and 'move-down':
41
42 '''python
43 [Python Domain Refinement Function Calls]
44 '''
45
46 With these corrected preconditions, the 'move-up' and
    'move-down' actions should now accurately reflect the
    natural language description of the Termes domain, and the
    error should be resolved.

```

Listing 9: Example of domain feedback from the Termes environment, where the LLM output refinement results into a correct domain.

B.4 Further experiment details

LLM calls per task. For each task in Algorithm 1, the overall complexity of LLM calls is $O(n_p \times n_d \times c_{max} + N)$. The complexity of domain generation is $O(n_p \times c_{max} \times n_d)$. This is because at first, n_p problem candidates are generated and for each problem candidate the algorithm goes through a refinement procedure (lines 1 and 2 of Algorithm 1). The refinement is a tree with depth c_{max} (where c_{max} is the maximum number of refinement turns) (line 5), and at each level of the tree, one node is expanded with n_d children (where n_d is the number of domain refinement candidates) (line 6), which leads to $O(n_p \times c_{max} \times n_d)$ complexity. Once the domain is ready, the complexity of task generation for N tasks is $O(N)$ since for each task we only call the LLM once to get a problem translation.

Number of successful seeds. In Table 2, we report the results over four seeds. To provide further analysis, we report the number of seeds a domain was successful in successfully generating a correct domain. The number of seeds that succeed in generating correct domain for the Termes, Movie, Miconic, Grippers, Hiking, Grippers-ood, and Floortile, are 4, 3, 3, 3, 2, 1, 1, respectively.

C Natural Language Description Generation

To generate natural language description of domains, problems, and environment states, we use the following strategies:

- **Domain:** We use a few-shot translation strategy. We first pick three diverse environments of “Grippers”, “Childsnack”, and “Termes” to manually (with assistance of GPT-4) curate domain translation. Then, we use these three domains as three-shot in-context examples to translate the rest of domains. The example prompt is provided in Listing 10.
- **Problems:** We use a similar few-shot translation strategy for problem translation. We first pick two diverse environments of “Termes” and “Satellite” for problem two-shot problem translation. Once one problem from a target domain is translated, we use that problem translation as in-context example to translate the rest of the problems. This step is crucial to ensure all problems from the same domain are translated in a consistent manner. The example prompt is provided in Listing 11.
- **Natural Language Predicate Description:** To generate natural language description of states, we generate a python files for each domain, with one function to produce natural language description of predicates for state description. The example prompt is provided in Listing 12.

```

1 Your task is to translate PDDL files into natural language.
2 Ensure that the resulting text covers natural language
  description of its actions, their preconditions, and
  effects.

```

```

3 DO NOT translate the problem PDDL files, only use problem PDDL
  to understand the domain. ALWAYS wrap your code in the
  appropriate markdown syntax.
4 Two examples are provided below.
5 Q:
6 Domain PDDL:
7 ```pddl
8 (define (domain gripper-strips)
9   (:requirements :strips :typing)
10  (:types room obj robot gripper)
11  (:predicates (at-robby ?r - robot ?x - room)
12               (at ?o - obj ?x - room)
13               (free ?r - robot ?g - gripper)
14               (carry ?r - robot ?o - obj ?g - gripper))
15  (:action move
16    :parameters (?r - robot ?from ?to - room)
17    :precondition (and (at-robby ?r ?from))
18    :effect (and (at-robby ?r ?to)
19                (not (at-robby ?r ?from))))
20  (:action pick
21    :parameters (?r - robot ?obj - obj ?room - room ?g -
22                gripper)
23    :precondition (and (at ?obj ?room) (at-robby ?r
24                      ?room) (free ?r ?g))
25    :effect (and (carry ?r ?obj ?g)
26                (not (at ?obj ?room))
27                (not (free ?r ?g))))
28  (:action drop
29    :parameters (?r - robot ?obj - obj ?room - room ?g -
30                gripper)
31    :precondition (and (carry ?r ?obj ?g) (at-robby ?r
32                      ?room))
33    :effect (and (at ?obj ?room)
34                (free ?r ?g)
35                (not (carry ?r ?obj ?g))))
36  ```
37 Problem PDDL:
38 ```pddl
39 (define (problem gripper-2-4-6)
40   (:domain gripper-strips)
41   (:objects robot1 robot2 - robot
42             rgripper1 lgripper1 rgripper2 lgripper2 - gripper
43             room1 room2 room3 room4 - room
44             ball1 ball2 ball3 ball4 ball5 ball6 - obj)
45   (:init
46     (at-robby robot1 room2)
47     (free robot1 rgripper1)
48     (free robot1 lgripper1)
49     (at-robby robot2 room3)
50     (free robot2 rgripper2)
51     (free robot2 lgripper2)
52     (at ball1 room3)
53     (at ball2 room1)
54     (at ball3 room3)

```

```

51 (at ball4 room2)
52 (at ball5 room4)
53 (at ball6 room4)
54 )
55 (:goal
56 (and
57 (at ball1 room4)
58 (at ball2 room1)
59 (at ball3 room1)
60 (at ball4 room2)
61 (at ball5 room1)
62 (at ball6 room1)
63 )
64 )
65 )
66 ```
67 A:
68 ```markdown
69 The gripper domain involves a world with multiple rooms,
    robots, and objects (balls). Each robot has two grippers
    that can be used to pick up and drop objects. The goal is
    to move objects from their initial locations to the
    desired goal locations using the robots and their grippers.
70 The domain includes three actions:
71 1. move: This action allows a robot to move from one room to
    another. The precondition is that the robot must be in the
    starting room. The effect is that the robot is no longer
    in the starting room and is now in the destination room.
72 2. pick: This action allows a robot to pick up an object using
    one of its grippers. The preconditions are that the object
    and the robot must be in the same room, and the specified
    gripper must be free (not holding any object). The effect
    is that the robot is now carrying the object with the
    specified gripper, the object is no longer in the room,
    and the gripper is no longer free.
73 3. drop: This action allows a robot to drop an object it is
    carrying in a specific room using one of its grippers. The
    preconditions are that the robot must be carrying the
    object with the specified gripper and the robot must be in
    the specified room. The effect is that the object is now
    in the room, the gripper is free, and the robot is no
    longer carrying the object with that gripper.
74 ```
75 Q:
76 Domain PDDL:
77 ```pddl
78 (define (domain child-snack)
79 (:requirements :typing :equality)
80 (:types child bread-portion content-portion sandwich tray
    place)
81 (:constants kitchen - place)
82 (:predicates (at_kitchen_bread ?b - bread-portion)
83               (at_kitchen_content ?c - content-portion)
84               (at_kitchen_sandwich ?s - sandwich)

```

```

85         (no_gluten_bread ?b - bread-portion)
86         (no_gluten_content ?c - content-portion)
87         (ontray ?s - sandwich ?t - tray)
88         (no_gluten_sandwich ?s - sandwich)
89         (allergic_gluten ?c - child)
90         (not_allergic_gluten ?c - child)
91         (served ?c - child)
92         (waiting ?c - child ?p - place)
93         (at ?t - tray ?p - place)
94         (notexist ?s - sandwich)
95     )
96 (:action make_sandwich_no_gluten
97   :parameters (?s - sandwich ?b - bread-portion ?c -
98     content-portion)
99   :precondition (and (at_kitchen_bread ?b)
100     (at_kitchen_content ?c)
101     (no_gluten_bread ?b)
102     (no_gluten_content ?c)
103     (notexist ?s))
104   :effect (and
105     (not (at_kitchen_bread ?b))
106     (not (at_kitchen_content ?c))
107     (at_kitchen_sandwich ?s)
108     (no_gluten_sandwich ?s)
109     (not (notexist ?s))
110   ))
111 (:action make_sandwich
112   :parameters (?s - sandwich ?b - bread-portion ?c -
113     content-portion)
114   :precondition (and (at_kitchen_bread ?b)
115     (at_kitchen_content ?c)
116     (notexist ?s)
117   )
118   :effect (and
119     (not (at_kitchen_bread ?b))
120     (not (at_kitchen_content ?c))
121     (at_kitchen_sandwich ?s)
122     (not (notexist ?s))
123   ))
124 (:action put_on_tray
125   :parameters (?s - sandwich ?t - tray)
126   :precondition (and (at_kitchen_sandwich ?s)
127     (at ?t kitchen))
128   :effect (and
129     (not (at_kitchen_sandwich ?s))
130     (ontray ?s ?t)))
131 (:action serve_sandwich_no_gluten
132   :parameters (?s - sandwich ?c - child ?t - tray ?p - place)
133   :precondition (and
134     (allergic_gluten ?c)
135     (ontray ?s ?t)
136     (waiting ?c ?p)
137     (no_gluten_sandwich ?s)
138     (at ?t ?p))

```

```

137         )
138     :effect (and (not (ontray ?s ?t))
139                 (served ?c)))
140 (:action serve_sandwich
141   :parameters (?s - sandwich ?c - child ?t - tray ?p - place)
142   :precondition (and (not_allergic_gluten ?c)
143                      (waiting ?c ?p)
144                      (ontray ?s ?t)
145                      (at ?t ?p))
146   :effect (and (not (ontray ?s ?t))
147               (served ?c)))
148 (:action move_tray
149   :parameters (?t - tray ?p1 ?p2 - place)
150   :precondition (and (at ?t ?p1))
151   :effect (and (not (at ?t ?p1))
152               (at ?t ?p2)))
153
154 )
155 '''
156 Problem PDDL:
157 '''pddl
158 ; child-snack task with 6 children and 0.4 gluten factor
159 ; constant factor of 1.3
160 ; random seed: 234324
161 (define (problem prob-snack)
162   (:domain child-snack)
163   (:objects
164     child1 child2 child3 child4 child5 child6 - child
165     bread1 bread2 bread3 bread4 bread5 bread6 - bread-portion
166     content1 content2 content3 content4 content5 content6 -
167       content-portion
168     tray1 tray2 - tray
169     table1 table2 table3 - place
170     sandw1 sandw2 sandw3 sandw4 sandw5 sandw6 sandw7 sandw8 -
171       sandwich
172   )
173   (:init
174     (at tray1 kitchen)
175     (at tray2 kitchen)
176     (at_kitchen_bread bread1)
177     (at_kitchen_bread bread2)
178     (at_kitchen_bread bread3)
179     (at_kitchen_bread bread4)
180     (at_kitchen_bread bread5)
181     (at_kitchen_bread bread6)
182     (at_kitchen_content content1)
183     (at_kitchen_content content2)
184     (at_kitchen_content content3)
185     (at_kitchen_content content4)
186     (at_kitchen_content content5)
187     (at_kitchen_content content6)
188     (no_gluten_bread bread2)
189     (no_gluten_bread bread5)
190     (no_gluten_content content3)

```

```

189     (no_gluten_content content6)
190     (allergic_gluten child1)
191     (allergic_gluten child5)
192     (not_allergic_gluten child2)
193     (not_allergic_gluten child3)
194     (not_allergic_gluten child4)
195     (not_allergic_gluten child6)
196     (waiting child1 table2)
197     (waiting child2 table1)
198     (waiting child3 table3)
199     (waiting child4 table2)
200     (waiting child5 table1)
201     (waiting child6 table1)
202     (notexist sandw1)
203     (notexist sandw2)
204     (notexist sandw3)
205     (notexist sandw4)
206     (notexist sandw5)
207     (notexist sandw6)
208     (notexist sandw7)
209     (notexist sandw8)
210 )
211 (:goal
212   (and
213     (served child1)
214     (served child2)
215     (served child3)
216     (served child4)
217     (served child5)
218     (served child6)
219   )
220 )
221 )
222 ```
223 A:
224 ```markdown
225 The child-snack domain is designed to plan the preparation and
    serving of sandwiches to children, some of whom have
    gluten allergies. The domain includes actions for making
    sandwiches with and without gluten, placing sandwiches on
    trays, moving trays between places, and serving the
    sandwiches to the children.
226 The actions defined in this domain include:
227 1. make_sandwich_no_gluten: This action is for making a
    gluten-free sandwich. The preconditions for this action
    are that there must be gluten-free bread and gluten-free
    content available in the kitchen, and the sandwich must
    not already exist. The effect of this action is that the
    gluten-free bread and content are no longer available in
    the kitchen, and a gluten-free sandwich is now available
    in the kitchen.
228 2. make_sandwich: This action is for making a regular sandwich
    that may contain gluten. The preconditions are that there
    must be bread and content available in the kitchen, and

```

```

the sandwich must not already exist. The effect of this
action is that the bread and content are no longer
available in the kitchen, and a sandwich is now available
in the kitchen.
229 3. put_on_tray: This action is for placing a sandwich on a
tray. The preconditions are that the sandwich must be in
the kitchen and the tray must be at the kitchen. The
effect is that the sandwich is no longer in the kitchen
and is now on the tray.
230 4. serve_sandwich_no_gluten: This action is for serving a
gluten-free sandwich to a child who is allergic to gluten.
The preconditions are that the child is allergic to
gluten, waiting at a place, and there is a gluten-free
sandwich on a tray at the same place. The effect is that
the sandwich is no longer on the tray and the child is
served.
231 5. serve_sandwich: This action is for serving a sandwich to a
child who is not allergic to gluten. The preconditions are
that the child is not allergic to gluten, waiting at a
place, and there is a sandwich on a tray at the same
place. The effect is that the sandwich is no longer on the
tray and the child is served.
232 6. move_tray: This action allows a tray to be moved from one
place to another. The precondition is that the tray is at
the starting place. The effect is that the tray is no
longer at the starting place and is now at the destination
place.
233 '''
234 Q:
235 Domain PDDL:
236 '''pddl
237 (define (domain termes)
238 (:requirements :typing :negative-preconditions)
239 (:types
240   numb - object
241   position - object
242 )
243 (:predicates
244   (height ?p - position ?h - numb)
245   (at ?p - position)
246   (has-block)
247   ;
248   ; static predicates
249   (SUCC ?n1 - numb ?n2 - numb)
250   (NEIGHBOR ?p1 - position ?p2 - position)
251   (IS-DEPOT ?p - position)
252 )
253 (:action move
254   :parameters (?from - position ?to - position ?h - numb)
255   :precondition
256   (and
257     (at ?from)
258     (NEIGHBOR ?from ?to)
259     (height ?from ?h)

```

```

260         (height ?to ?h)
261     )
262     :effect
263     (and
264         (not (at ?from))
265         (at ?to)
266     )
267 )
268 (:action move-up
269     :parameters (?from - position ?hfrom - numb ?to - position
270                 ?hto - numb)
271     :precondition
272     (and
273         (at ?from)
274         (NEIGHBOR ?from ?to)
275         (height ?from ?hfrom)
276         (height ?to ?hto)
277         (SUCC ?hto ?hfrom)
278     )
279     :effect
280     (and
281         (not (at ?from))
282         (at ?to)
283     )
284 )
285 (:action move-down
286     :parameters (?from - position ?hfrom - numb ?to - position
287                 ?hto - numb)
288     :precondition
289     (and
290         (at ?from)
291         (NEIGHBOR ?from ?to)
292         (height ?from ?hfrom)
293         (height ?to ?hto)
294         (SUCC ?hfrom ?hto)
295     )
296     :effect
297     (and
298         (not (at ?from))
299         (at ?to)
300     )
301 )
302 (:action place-block
303     :parameters (?rpos - position ?bpos - position ?hbefore -
304                 numb ?hafter - numb)
305     :precondition
306     (and
307         (at ?rpos)
308         (NEIGHBOR ?rpos ?bpos)
309         (height ?rpos ?hbefore)
310         (height ?bpos ?hbefore)
311         (SUCC ?hafter ?hbefore)
312         (has-block)
313         (not (IS-DEPOT ?bpos))

```

```

311 )
312 :effect
313 (and
314     (not (height ?bpos ?hbefore))
315     (height ?bpos ?hafter)
316     (not (has-block))
317 )
318 )
319 (:action remove-block
320     :parameters (?rpos - position ?bpos - position ?hbefore -
321         numb ?hafter - numb)
322     :precondition
323     (and
324         (at ?rpos)
325         (NEIGHBOR ?rpos ?bpos)
326         (height ?rpos ?hafter)
327         (height ?bpos ?hbefore)
328         (SUCC ?hbefore ?hafter)
329         (not (has-block))
330     )
331     :effect
332     (and
333         (not (height ?bpos ?hbefore))
334         (height ?bpos ?hafter)
335         (has-block)
336     )
337 )
338 (:action create-block
339     :parameters (?p - position)
340     :precondition
341     (and
342         (at ?p)
343         (not (has-block))
344         (IS-DEPOT ?p)
345     )
346     :effect
347     (and
348         (has-block)
349     )
350 )
351 (:action destroy-block
352     :parameters (?p - position)
353     :precondition
354     (and
355         (at ?p)
356         (has-block)
357         (IS-DEPOT ?p)
358     )
359     :effect
360     (and
361         (not (has-block))
362     )
363 )

```

```

364  """
365  Problem PDDL:
366  """pddl
367  (define (problem prob)
368  (:domain termes)
369  ; Initial state:
370  ; 0 0 ROD
371  ; 0 0 0
372  ; 0 0 0
373  ; Goal state:
374  ; 0 0 0
375  ; 0 1 0
376  ; 0 0 0
377  ; Maximal height: 1
378  (:objects
379     n0 - numb
380     n1 - numb
381     pos-0-0 - position
382     pos-0-1 - position
383     pos-0-2 - position
384     pos-1-0 - position
385     pos-1-1 - position
386     pos-1-2 - position
387     pos-2-0 - position
388     pos-2-1 - position
389     pos-2-2 - position
390  )
391  (:init
392     (height pos-0-0 n0)
393     (height pos-0-1 n0)
394     (height pos-0-2 n0)
395     (height pos-1-0 n0)
396     (height pos-1-1 n0)
397     (height pos-1-2 n0)
398     (height pos-2-0 n0)
399     (height pos-2-1 n0)
400     (height pos-2-2 n0)
401     (at pos-2-0)
402     (SUCC n1 n0)
403     (NEIGHBOR pos-0-0 pos-1-0)
404     (NEIGHBOR pos-0-0 pos-0-1)
405     (NEIGHBOR pos-0-1 pos-1-1)
406     (NEIGHBOR pos-0-1 pos-0-0)
407     (NEIGHBOR pos-0-1 pos-0-2)
408     (NEIGHBOR pos-0-2 pos-1-2)
409     (NEIGHBOR pos-0-2 pos-0-1)
410     (NEIGHBOR pos-1-0 pos-0-0)
411     (NEIGHBOR pos-1-0 pos-2-0)
412     (NEIGHBOR pos-1-0 pos-1-1)
413     (NEIGHBOR pos-1-1 pos-0-1)
414     (NEIGHBOR pos-1-1 pos-2-1)
415     (NEIGHBOR pos-1-1 pos-1-0)
416     (NEIGHBOR pos-1-1 pos-1-2)
417     (NEIGHBOR pos-1-2 pos-0-2)

```

```

418 (NEIGHBOR pos-1-2 pos-2-2)
419 (NEIGHBOR pos-1-2 pos-1-1)
420 (NEIGHBOR pos-2-0 pos-1-0)
421 (NEIGHBOR pos-2-0 pos-2-1)
422 (NEIGHBOR pos-2-1 pos-1-1)
423 (NEIGHBOR pos-2-1 pos-2-0)
424 (NEIGHBOR pos-2-1 pos-2-2)
425 (NEIGHBOR pos-2-2 pos-1-2)
426 (NEIGHBOR pos-2-2 pos-2-1)
427 (IS-DEPOT pos-2-0)
428 )
429 (:goal
430 (and
431 (height pos-0-0 n0)
432 (height pos-0-1 n0)
433 (height pos-0-2 n0)
434 (height pos-1-0 n0)
435 (height pos-1-1 n1)
436 (height pos-1-2 n0)
437 (height pos-2-0 n0)
438 (height pos-2-1 n0)
439 (height pos-2-2 n0)
440 (not (has-block))
441 )
442 )
443 )
444 ```
445 A:
446 ```markdown
447 The Termes domain is a planning domain that simulates the
    behavior of robotic agents (inspired by termites) that can
    move around, pick up blocks, stack them to build
    structures, and remove blocks from structures. The domain
    includes actions for moving the robot, placing and
    removing blocks, and creating and destroying blocks at a
    depot.
448 The actions defined in this domain include:
449 1. move: This action allows the robot to move from one
    position to another at the same height. The preconditions
    are that the robot is at the starting position, the
    starting position is a neighbor to the destination
    position, and both positions have the same height. The
    effect is that the robot is no longer at the starting
    position and is now at the destination position.
450 2. move-up: This action allows the robot to move from a lower
    position to a neighboring higher position. The
    preconditions are that the robot is at the starting
    position, the starting position is a neighbor to the
    destination position, the starting position has a certain
    height, and the destination position's height is one less
    than the starting position's height. The effect is that
    the robot is no longer at the starting position and is now
    at the destination position.

```

```

451 3. move-down: This action allows the robot to move from a
    higher position to a neighboring lower position. The
    preconditions are that the robot is at the starting
    position, the starting position is a neighbor to the
    destination position, the starting position has a certain
    height, and the destination position's height is one less
    than the starting position's height. The effect is that
    the robot is no longer at the starting position and is now
    at the destination position.
452 4. place-block: This action allows the robot to place a block
    at a neighboring position, increasing the height of that
    position by one. The preconditions are that the robot is
    at a position next to the block position, both positions
    have the same height, the robot has a block, and the block
    position is not a depot. The effect is that the height of
    the block position is increased by one, and the robot no
    longer has a block.
453 5. remove-block: This action allows the robot to remove a
    block from a neighboring position, decreasing the height
    of that position by one. The preconditions are that the
    robot is at a position next to the block position, the
    robot's position is one height unit higher than the block
    position, and the robot does not have a block. The effect
    is that the height of the block position is decreased by
    one, and the robot now has a block.
454 6. create-block: This action allows the robot to create a
    block at a depot. The preconditions are that the robot is
    at the depot and does not have a block. The effect is that
    the robot now has a block.
455 7. destroy-block: This action allows the robot to destroy a
    block at a depot. The preconditions are that the robot is
    at the depot and has a block. The effect is that the robot
    no longer has a block.
456 '' '
457 Q:
458 Domain PDDL:
459 '' 'pddl
460 [Target Domain PDDL Code]
461 '' '
462 Problem PDDL:
463 '' 'pddl
464 [Target Problem PDDL Code]
465 '' '

```

Listing 10: Domain back-translation prompt template, with domain PDDL and problem PDDL placeholders for each target domain. Some PDDL credit comments are omitted for clarity.

```

1 Your task is to translate problem PDDL files into natural
  language. Ensure that the resulting description covers all
  initial state and goal conditions.
2 DO NOT be lazy in your response, be extremely precise in your
  descriptions such that all conditions are covered in your
  description and there is no ambiguity in your description.
3 If you do not find any common rule about some conditions, list
  all of them.

```

```

4 For the initial conditions, start with "Initially:", and for
  the goal conditions, start with "Your goal is to".
5 ALWAYS wrap your code in the appropriate markdown syntax.
6 Two examples are provided below.
7 Q:
8 Domain Description:
9 ```markdown
10 The Termes domain is a planning domain that simulates the
  behavior of robotic agents (inspired by termites) that can
  move around, pick up blocks, stack them to build
  structures, and remove blocks from structures. The domain
  includes actions for moving the robot, placing and
  removing blocks, and creating and destroying blocks at a
  depot.
11 The actions defined in this domain include:
12 1. move: This action allows the robot to move from one
  position to another at the same height. The preconditions
  are that the robot is at the starting position, the
  starting position is a neighbor to the destination
  position, and both positions have the same height. The
  effect is that the robot is no longer at the starting
  position and is now at the destination position.
13 2. move-up: This action allows the robot to move from a lower
  position to a neighboring higher position. The
  preconditions are that the robot is at the starting
  position, the starting position is a neighbor to the
  destination position, the starting position has a certain
  height, and the destination position's height is one less
  than the starting position's height. The effect is that
  the robot is no longer at the starting position and is now
  at the destination position.
14 3. move-down: This action allows the robot to move from a
  higher position to a neighboring lower position. The
  preconditions are that the robot is at the starting
  position, the starting position is a neighbor to the
  destination position, the starting position has a certain
  height, and the destination position's height is one less
  than the starting position's height. The effect is that
  the robot is no longer at the starting position and is now
  at the destination position.
15 4. place-block: This action allows the robot to place a block
  at a neighboring position, increasing the height of that
  position by one. The preconditions are that the robot is
  at a position next to the block position, both positions
  have the same height, the robot has a block, and the block
  position is not a depot. The effect is that the height of
  the block position is increased by one, and the robot no
  longer has a block.
16 5. remove-block: This action allows the robot to remove a
  block from a neighboring position, decreasing the height
  of that position by one. The preconditions are that the
  robot is at a position next to the block position, the
  robot's position is one height unit higher than the block
  position, and the robot does not have a block. The effect

```

```

is that the height of the block position is decreased by
one, and the robot now has a block.
17 6. create-block: This action allows the robot to create a
    block at a depot. The preconditions are that the robot is
    at the depot and does not have a block. The effect is that
    the robot now has a block.
18 7. destroy-block: This action allows the robot to destroy a
    block at a depot. The preconditions are that the robot is
    at the depot and has a block. The effect is that the robot
    no longer has a block.
19 ```
20 Problem PDDL:
21 ```pddl
22 (define (problem prob)
23 (:domain termes)
24 ; Initial state:
25 ; 0 0 ROD
26 ; 0 0 0
27 ; 0 0 0
28 ; Goal state:
29 ; 0 0 0
30 ; 0 1 0
31 ; 0 0 0
32 ; Maximal height: 1
33 (:objects
34   n0 - numb
35   n1 - numb
36   pos-0-0 - position
37   pos-0-1 - position
38   pos-0-2 - position
39   pos-1-0 - position
40   pos-1-1 - position
41   pos-1-2 - position
42   pos-2-0 - position
43   pos-2-1 - position
44   pos-2-2 - position
45 )
46 (:init
47   (height pos-0-0 n0)
48   (height pos-0-1 n0)
49   (height pos-0-2 n0)
50   (height pos-1-0 n0)
51   (height pos-1-1 n0)
52   (height pos-1-2 n0)
53   (height pos-2-0 n0)
54   (height pos-2-1 n0)
55   (height pos-2-2 n0)
56   (at pos-2-0)
57   (SUCC n1 n0)
58   (NEIGHBOR pos-0-0 pos-1-0)
59   (NEIGHBOR pos-0-0 pos-0-1)
60   (NEIGHBOR pos-0-1 pos-1-1)
61   (NEIGHBOR pos-0-1 pos-0-0)
62   (NEIGHBOR pos-0-1 pos-0-2)

```

```

63     (NEIGHBOR pos-0-2 pos-1-2)
64     (NEIGHBOR pos-0-2 pos-0-1)
65     (NEIGHBOR pos-1-0 pos-0-0)
66     (NEIGHBOR pos-1-0 pos-2-0)
67     (NEIGHBOR pos-1-0 pos-1-1)
68     (NEIGHBOR pos-1-1 pos-0-1)
69     (NEIGHBOR pos-1-1 pos-2-1)
70     (NEIGHBOR pos-1-1 pos-1-0)
71     (NEIGHBOR pos-1-1 pos-1-2)
72     (NEIGHBOR pos-1-2 pos-0-2)
73     (NEIGHBOR pos-1-2 pos-2-2)
74     (NEIGHBOR pos-1-2 pos-1-1)
75     (NEIGHBOR pos-2-0 pos-1-0)
76     (NEIGHBOR pos-2-0 pos-2-1)
77     (NEIGHBOR pos-2-1 pos-1-1)
78     (NEIGHBOR pos-2-1 pos-2-0)
79     (NEIGHBOR pos-2-1 pos-2-2)
80     (NEIGHBOR pos-2-2 pos-1-2)
81     (NEIGHBOR pos-2-2 pos-2-1)
82     (IS-DEPOT pos-2-0)
83 )
84 (:goal
85 (and
86   (height pos-0-0 n0)
87   (height pos-0-1 n0)
88   (height pos-0-2 n0)
89   (height pos-1-0 n0)
90   (height pos-1-1 n1)
91   (height pos-1-2 n0)
92   (height pos-2-0 n0)
93   (height pos-2-1 n0)
94   (height pos-2-2 n0)
95   (not (has-block)))
96 )
97 )
98 )
99 ‘‘‘
100 A:
101 ‘‘‘markdown
102 You control a robot capable of building structures by moving
    and manipulating blocks.
103 The environment consists of a grid of positions and two
    heights, numbered 0 and 1. The robot can move at the same
    height, move up one height, or move down one height. It
    can also place or remove a block at a neighboring
    position, or create or destroy a block at the depot. A
    block’s height increases by one when placed and decreases
    by one when removed.
104 Initially:
105 - All positions on the grid have a height of 0.
106 - The robot is at position pos-2-0, which is the depot.
107 - The robot does not have a block.
108 - The positions have the following neighboring relationships:
109   - pos-0-0 neighbors pos-1-0 and pos-0-1

```

```

110 - pos-0-1 neighbors pos-1-1, pos-0-0, and pos-0-2
111 - pos-0-2 neighbors pos-1-2 and pos-0-1
112 - pos-1-0 neighbors pos-0-0, pos-2-0, and pos-1-1
113 - pos-1-1 neighbors pos-0-1, pos-2-1, pos-1-0, and pos-1-2
114 - pos-1-2 neighbors pos-0-2, pos-2-2, and pos-1-1
115 - pos-2-0 neighbors pos-1-0 and pos-2-1, and is the depot
116 - pos-2-1 neighbors pos-1-1, pos-2-0, and pos-2-2
117 - pos-2-2 neighbors pos-1-2 and pos-2-1
118 - There is a successor relationship between the numbers n1 and
    n0.
119 Your goal is to achieve the following configuration:
120 - The height at pos-1-1 needs to be 1.
121 - All other positions must remain at height 0.
122 - The robot should not have a block at the end of the task.
123 ```
124 Q:
125 Domain Description:
126 ```markdown
127 The satellite domain is designed to model the operation of
    satellites that can take images of various targets in
    different modes. Each satellite is equipped with
    instruments that can be turned on and off, calibrated, and
    used to take images. The domain includes actions for
    turning the satellite to point at different directions,
    switching instruments on and off, calibrating instruments,
    and taking images.
128 The actions defined in this domain include:
129 1. turn_to: This action changes the direction the satellite is
    pointing. The preconditions are that the satellite must be
    pointing at a previous direction, and both the new and
    previous directions are valid. The effect is that the
    satellite is now pointing at the new direction and no
    longer pointing at the previous direction.
130 2. switch_on: This action turns on an instrument on board the
    satellite. The preconditions are that the instrument must
    be on board the satellite and there must be power
    available on the satellite. The effect is that the
    instrument is powered on, it is no longer calibrated, and
    the satellite no longer has power available.
131 3. switch_off: This action turns off an instrument on board
    the satellite. The preconditions are that the instrument
    must be on board the satellite and it must be powered on.
    The effect is that the satellite has power available and
    the instrument is no longer powered on.
132 4. calibrate: This action calibrates an instrument on board
    the satellite. The preconditions are that the satellite
    must be pointing at a calibration target for the
    instrument, the instrument must be on board the satellite
    and powered on. The effect is that the instrument is
    calibrated.
133 5. take_image: This action uses an instrument on board the
    satellite to take an image in a specific mode of a
    direction the satellite is pointing at. The preconditions
    are that the satellite must be pointing at the direction,

```

```

    the instrument must be calibrated, on board the satellite,
    support the mode, and be powered on. The effect is that an
    image of the direction in the specific mode is now
    available.
134  """
135  Problem PDDL:
136  """pddl
137  (define (problem strips-sat-x-1)
138  (:domain satellite)
139  (:objects
140    satellite0
141    instrument0
142    satellite1
143    instrument1
144    instrument2
145    instrument3
146    satellite2
147    instrument4
148    instrument5
149    instrument6
150    satellite3
151    instrument7
152    satellite4
153    instrument8
154    thermograph2
155    image3
156    infrared1
157    spectrograph4
158    infrared0
159    Star1
160    Star4
161    Star0
162    GroundStation3
163    Star2
164    Star5
165    Planet6
166    Phenomenon7
167    Star8
168    Phenomenon9
169    Star10
170    Star11
171    Star12
172    Planet13
173    Planet14
174    Phenomenon15
175    Planet16
176    Star17
177    Star18
178    Planet19
179  )
180  (:init
181    (satellite satellite0)
182    (instrument instrument0)
183    (supports instrument0 spectrograph4)

```

```

184 (calibration_target instrument0 Star0)
185 (on_board instrument0 satellite0)
186 (power_avail satellite0)
187 (pointing satellite0 Star8)
188 (satellite satellite1)
189 (instrument instrument1)
190 (supports instrument1 infrared0)
191 (supports instrument1 infrared1)
192 (calibration_target instrument1 GroundStation3)
193 (instrument instrument2)
194 (supports instrument2 infrared1)
195 (supports instrument2 infrared0)
196 (calibration_target instrument2 Star2)
197 (instrument instrument3)
198 (supports instrument3 spectrograph4)
199 (supports instrument3 infrared1)
200 (supports instrument3 thermograph2)
201 (calibration_target instrument3 Star0)
202 (on_board instrument1 satellite1)
203 (on_board instrument2 satellite1)
204 (on_board instrument3 satellite1)
205 (power_avail satellite1)
206 (pointing satellite1 GroundStation3)
207 (satellite satellite2)
208 (instrument instrument4)
209 (supports instrument4 infrared1)
210 (supports instrument4 image3)
211 (supports instrument4 infrared0)
212 (calibration_target instrument4 Star2)
213 (instrument instrument5)
214 (supports instrument5 thermograph2)
215 (supports instrument5 spectrograph4)
216 (calibration_target instrument5 Star0)
217 (instrument instrument6)
218 (supports instrument6 infrared0)
219 (calibration_target instrument6 GroundStation3)
220 (on_board instrument4 satellite2)
221 (on_board instrument5 satellite2)
222 (on_board instrument6 satellite2)
223 (power_avail satellite2)
224 (pointing satellite2 Star4)
225 (satellite satellite3)
226 (instrument instrument7)
227 (supports instrument7 image3)
228 (calibration_target instrument7 Star2)
229 (on_board instrument7 satellite3)
230 (power_avail satellite3)
231 (pointing satellite3 Phenomenon9)
232 (satellite satellite4)
233 (instrument instrument8)
234 (supports instrument8 infrared0)
235 (supports instrument8 spectrograph4)
236 (supports instrument8 infrared1)
237 (calibration_target instrument8 Star2)

```

```

238 (on_board instrument8 satellite4)
239 (power_avail satellite4)
240 (pointing satellite4 Phenomenon9)
241 (mode thermograph2)
242 (mode image3)
243 (mode infrared1)
244 (mode spectrograph4)
245 (mode infrared0)
246 (direction Star1)
247 (direction Star4)
248 (direction Star0)
249 (direction GroundStation3)
250 (direction Star2)
251 (direction Star5)
252 (direction Planet6)
253 (direction Phenomenon7)
254 (direction Star8)
255 (direction Phenomenon9)
256 (direction Star10)
257 (direction Star11)
258 (direction Star12)
259 (direction Planet13)
260 (direction Planet14)
261 (direction Phenomenon15)
262 (direction Planet16)
263 (direction Star17)
264 (direction Star18)
265 (direction Planet19)
266 )
267 (:goal (and
268 (pointing satellite0 Phenomenon9)
269 (pointing satellite1 Star4)
270 (pointing satellite4 Star11)
271 (have_image Star5 image3)
272 (have_image Planet6 infrared1)
273 (have_image Phenomenon7 infrared1)
274 (have_image Star8 image3)
275 (have_image Star10 thermograph2)
276 (have_image Star11 infrared1)
277 (have_image Planet13 spectrograph4)
278 (have_image Planet14 thermograph2)
279 (have_image Phenomenon15 infrared0)
280 (have_image Planet16 image3)
281 (have_image Star17 infrared0)
282 ))
283 )
284 ‘‘‘
285 A:
286 ‘‘‘markdown
287 You are operating a constellation of satellites tasked with
    taking images of various targets in different modes.
288 Initially:
289 - There are five satellites (satellite0 to satellite4) with
    corresponding instruments (instrument0 to instrument8).

```

```

290 - Each instrument supports specific modes and has a
    calibration target:
291 - Instrument0 supports spectrograph4 and targets Star0.
292 - Instrument1 supports infrared0 and infrared1, targeting
    GroundStation3.
293 - Instrument2 supports infrared1 and infrared0, targeting
    Star2.
294 - Instrument3 supports spectrograph4, infrared1, and
    thermograph2, targeting Star0.
295 - Instrument4 supports infrared1, image3, and infrared0,
    targeting Star2.
296 - Instrument5 supports thermograph2 and spectrograph4,
    targeting Star0.
297 - Instrument6 supports infrared0, targeting GroundStation3.
298 - Instrument7 supports image3, targeting Star2.
299 - Instrument8 supports infrared0, spectrograph4, and
    infrared1, targeting Star2.
300 - Instruments are on board their respective satellites, and
    all satellites have power available.
301 - Satellites are pointing at various directions:
302 - Satellite0 is pointing at Star8.
303 - Satellite1 is pointing at GroundStation3.
304 - Satellite2 is pointing at Star4.
305 - Satellite3 is pointing at Phenomenon9.
306 - Satellite4 is pointing at Phenomenon9.
307 - There are various modes (thermograph2, image3, infrared1,
    spectrograph4, infrared0) and directions (Star1 to Star18,
    GroundStation3, Planet6, Phenomenon7, Phenomenon9,
    Planet13, Planet14, Phenomenon15, Planet16, Planet19).
308 Your goal is to:
309 - Point satellite0 at Phenomenon9.
310 - Point satellite1 at Star4.
311 - Point satellite4 at Star11.
312 - Have images of the following targets in the specified modes:
313 - Star5 in image3 mode.
314 - Planet6 in infrared1 mode.
315 - Phenomenon7 in infrared1 mode.
316 - Star8 in image3 mode.
317 - Star10 in thermograph2 mode.
318 - Star11 in infrared1 mode.
319 - Planet13 in spectrograph4 mode.
320 - Planet14 in thermograph2 mode.
321 - Phenomenon15 in infrared0 mode.
322 - Planet16 in image3 mode.
323 - Star17 in infrared0 mode.
324 To achieve these goals, you will need to turn the satellites
    to point at the correct directions, switch on and
    calibrate the necessary instruments, and take images using
    the calibrated instruments in the supported modes.
325 ```
326 Q:
327 Domain Description:
328 ```markdown
329 [Target Domain Natural Language Description]

```

```

330 '''
331 Problem PDDL:
332 '''pddl
333 [Target Problem PDDL Code]
334 '''
335 A:

```

Listing 11: Domain back-translation prompt template, with domain natural language description and problem PDDL placeholders for each target domain.

```

1 Your task is to generate python predicate descriptor for each
  environment. You are given the natural language
  description of the domain along with the PDDL code.
2 Q:
3 Domain Description:
4 '''markdown
5 The robot has four actions: pickup, putdown, stack, and
  unstack. The domain assumes a world where there are a set
  of blocks that can be stacked on top of each other, an arm
  that can hold one block at a time, and a table where
  blocks can be placed.
6 The actions defined in this domain include:
7 pickup: allows the arm to pick up a block from the table if it
  is clear and the arm is empty. After the pickup action,
  the arm will be holding the block, and the block will no
  longer be on the table or clear.
8 putdown: allows the arm to put down a block on the table if it
  is holding a block. After the putdown action, the arm will
  be empty, and the block will be on the table and clear.
9 stack: allows the arm to stack a block on top of another block
  if the arm is holding the top block and the bottom block
  is clear. After the stack action, the arm will be empty,
  the top block will be on top of the bottom block, and the
  bottom block will no longer be clear.
10 unstack: allows the arm to unstack a block from on top of
  another block if the arm is empty and the top block is
  clear. After the unstack action, the arm will be holding
  the top block, the top block will no longer be on top of
  the bottom block, and the bottom block will be clear.
11 '''
12 Domain PDDL:
13 '''pddl
14 (define (domain blocksworld-4ops)
15   (:requirements :strips)
16   (:predicates (clear ?x)
17                 (on-table ?x)
18                 (arm-empty)
19                 (holding ?x)
20                 (on ?x ?y))
21   (:action pickup
22     :parameters (?ob)
23     :precondition (and (clear ?ob) (on-table ?ob) (arm-empty))
24     :effect (and (holding ?ob) (not (clear ?ob)) (not (on-table
25       ?ob))
26                (not (arm-empty))))

```

```

26 (:action putdown
27   :parameters (?ob)
28   :precondition (holding ?ob)
29   :effect (and (clear ?ob) (arm-empty) (on-table ?ob)
30              (not (holding ?ob))))
31 (:action stack
32   :parameters (?ob ?underob)
33   :precondition (and (clear ?underob) (holding ?ob))
34   :effect (and (arm-empty) (clear ?ob) (on ?ob ?underob)
35              (not (clear ?underob)) (not (holding ?ob))))
36 (:action unstack
37   :parameters (?ob ?underob)
38   :precondition (and (on ?ob ?underob) (clear ?ob) (arm-empty))
39   :effect (and (holding ?ob) (clear ?underob)
40              (not (on ?ob ?underob)) (not (clear ?ob)) (not
41              (arm-empty))))
42
43 A:
44 '''python
45 def describe_predicate(predicate_name, predicate_args):
46     """
47     Predicates:
48     - (clear ?x)
49     - (on-table ?x)
50     - (arm-empty)
51     - (holding ?x)
52     - (on ?x ?y)
53     :param predicate_name: str
54     :param predicate_args: Tuple[str]
55     :return: Tuple[str, str] - (positive, negative)
56     """
57     # (clear ?x)
58     if predicate_name == "clear":
59         (x,) = predicate_args
60         return f"Block {x} is clear.", f"Block {x} is not
61         clear."
62     # (on-table ?x)
63     elif predicate_name == "on-table":
64         (x,) = predicate_args
65         return f"Block {x} is on the table.", f"Block {x} is
66         not on the table."
67     # (arm-empty)
68     elif predicate_name == "arm-empty":
69         return "Arm is empty.", "Arm is not empty."
70     # (holding ?x)
71     elif predicate_name == "holding":
72         (x,) = predicate_args
73         return f"Arm is holding block {x}.", f"Arm is not
74         holding block {x}."
75     # (on ?x ?y)
76     elif predicate_name == "on":
77         (x, y) = predicate_args
78         return f"Block {x} is on block {y}.", f"Block {x} is
79         not on block {y}."

```

```

75     else:
76         raise ValueError(f"Unknown predicate:
77             {predicate_name}")
78     """
79     Q:
80     Domain Description:
81     """markdown
82     [Target Domain Natural Language Description]
83     """
84     Domain PDDL:
85     """pddl
86     [Target Domain PDDL Code]
87     """
88     A:

```

Listing 12: Predicate translation python code generation prompt.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [\[Yes\]](#)

Justification: The claims are supported by experiments.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: In the conclusion section, we discuss limitations of our work.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory Assumptions and Proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[NA\]](#)

Justification: Our paper does not include theoretical results.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental Result Reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: We describe our method in Algorithm 1, and Appendix B. The code will be made publicly available upon publication.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The code will be made publicly available upon publication. We explain our method in Algorithm 1. For the data, the PDDL files are publicly available, and we provide examples on how to obtain natural language descriptions.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental Setting/Details

Question: Does the paper specify all the training and test details (e.g., data splits, hyper-parameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: Our method uses pre-trained large language models, and we provide examples in Algorithm 1 on how we prompt the LLMs.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment Statistical Significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Figure 2b and Figure 4 show standard error. Table 2 does not contain any confidence intervals as the experiments become computationally expensive.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments Compute Resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: In the experiments section, we reported the number of tokens used to create Table 2.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code Of Ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: Yes

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader Impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [NA]

Justification: The setup used in our work does not have direct societal impact as it is centered around generating PDDL code.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: Our work is a fundamental work and poses no such risk.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We have cited the sources from which we use library, data, and code.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New Assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [NA]

Justification: The code will be made publicly available upon publication

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and Research with Human Subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.